

Resolución de problemas y algoritmos

Dra. Jessica Andrea Carballido

jessicarballido@gmail.com



Dpto. de Ciencias e Ingeniería de la Computación

UNIVERSIDAD NACIONAL DEL SUR

TRAZA FUNCION

Problema

- Dado un número, se desea convertirlo a vocal de acuerdo a su último dígito. Utilizar el siguiente cuadro para realizar la conversión:

Ultimo dígito del número:	Convertir a:
0, 1 o 2	A
3, 4 o 5	E
6	I
7 u 8	O
9	U

Implementar una función para resolverlo

```
function codificar(num: integer): char;  
var ult: integer;  
begin  
  ult:= num mod 10;  
  case ult of  
    0,1,2: codificar:='A';  
    3,4,5: codificar:='E';  
    6: codificar:='I';  
    7,8: codificar:='O';  
    9: codificar:='U';  
  end;  
end;
```

Problema

- Dada una secuencia de números positivos y su longitud, mostrar por pantalla el primer número que convertido con la primitiva anterior se transforma en una A.

Algoritmo NumAVOCAL

DE: secuencia y su longitud

DS: mostrar el 1er numero que convertido es A

Mientras no encuentro y hay números para leer

 leer numero

 hay un numero menos en la secuencia

 si convertido es A entonces ENCONTRE

Program AVOCAL;

Var CANT, elem: integer; ENCONTRE: boolean;

function codificar(num: integer): char;

Var ult: integer;

...

Begin

writeln('Ingrese la longitud de la secuencia: ');

readln(CANT);

ENCONTRE:=FALSE;

while ((CANT>0) and (ENCONTRE=FALSE)) do

begin

read(elem);

CANT:=CANT-1;

if codificar(elem)='A'

then ENCONTRE:=TRUE;

end;

writeln('El primer elem. q codificado es una A es: ', elem);

End.

¿Qué pasa si CANT = 0?

¿Qué pasa si no había ninguno? CONTROLARLO en una nueva versión

TRAZA

- Preparamos la memoria del programa principal con todas las variables declaradas para su bloque ejecutable (llamadas variables globales).
- En el ejemplo, la memoria del programa principal debe contener 3 variables: ELEM, CANT y ECONTRE.

ELEM	
CANT	
ENCONTRE	

Memoria del programa principal

ELEM	
CANT	
ENCONTRE	

Comenzamos a ejecutar una a una las instrucciones del programa principal.

```
Begin
writeln('Ingrese la longitud de la secuencia: ');
readln(CANT);
ENCONTRE:=FALSE;
writeln('Ingrese la secuencia');
while ((CANT>0) and (ENCONTRE=FALSE)) do
begin
    read(Elem);
    CANT:=CANT-1;
    if codificar(elem)='A'
        then ENCONTRE:=TRUE;
end;
writeln('El primer elem. q codificado es una A es: ', Elem);
End.
```

Ingrese la longitud de la secuencia:

Memoria del programa principal

ELEM	
CANT	5
ENCONTRE	

Comenzamos a ejecutar una a una las instrucciones del programa principal.

Begin

```
writeln('Ingrese la longitud de la secuencia: ');
```

```
readln(CANT);
```

```
ENCONTRE:=FALSE;
```

```
writeln('Ingrese la secuencia');
```

```
while ((CANT>0) and (ENCONTRE=FALSE)) do
```

```
begin
```

```
    read(Elem);
```

```
    CANT:=CANT-1;
```

```
    if codificar(elem)='A'
```

```
        then ENCONTRE:=TRUE;
```

```
end;
```

```
writeln('El primer elem. q codificado es una A es: ', Elem);
```

```
End.
```

Ingrese la longitud de la secuencia: 5

Memoria del programa principal

ELEM	
CANT	5
ENCONTRE	FALSE

Comenzamos a ejecutar una a una las instrucciones del programa principal.

Begin

```
writeln('Ingrese la longitud de la secuencia: ');
```

```
readln(CANT);
```

```
ENCONTRE:=FALSE;
```

```
writeln('Ingrese la secuencia');
```

```
while ((CANT>0) and (ENCONTRE=FALSE)) do
```

```
begin
```

```
    read(Elem);
```

```
    CANT:=CANT-1;
```

```
    if codificar(elem)='A'
```

```
        then ENCONTRE:=TRUE;
```

```
end;
```

```
writeln('El primer elem. q codificado es una A es: ', Elem);
```

```
End.
```

Ingrese la longitud de la secuencia: 5

Memoria del programa principal

ELEM	
CANT	5
ENCONTRE	FALSE

Comenzamos a ejecutar una a una las instrucciones del programa principal.

Begin

```
writeln('Ingrese la longitud de la secuencia: ');
```

```
readln(CANT);
```

```
ENCONTRE:=FALSE;
```

```
writeln('Ingrese la secuencia');
```

```
while ((CANT>0) and (ENCONTRE=FALSE)) do
```

```
begin
```

```
    read(Elem);
```

```
    CANT:=CANT-1;
```

```
    if codificar(elem)='A'
```

```
        then ENCONTRE:=TRUE;
```

```
end;
```

```
writeln('El primer elem. q codificado es una A es: ', Elem);
```

```
End.
```

Ingrese la longitud de la secuencia: 5
Ingrese la secuencia: 258 12 324 5 776

BUFFER

258

12

324

5

776

Memoria del programa principal

ELEM	
CANT	5
ENCONTRE	FALSE

Comenzamos a ejecutar una a una las instrucciones del programa principal.

Begin

writeln('Ingrese la longitud de la secuencia: ');

readln(CANT);

ENCONTRE:=FALSE;

writeln('Ingrese la secuencia');

while ((CANT>0) and (ENCONTRE=FALSE)) do

begin

 read(Elem);

 CANT:=CANT-1;

 if **codificar(elem)**='A'

 then ENCONTRE:=TRUE;

end;

writeln('El primer elem. q codificado es una A es: ', Elem);

End.

Las dos sub-expresiones son VERDADERAS

Ingrese la longitud de la secuencia: 5
Ingrese la secuencia: 258 12 324 5 776

BUFFER

258

12

324

5

776

Memoria del programa principal

ELEM	258
CANT	5
ENCONTRE	FALSE

Comenzamos a ejecutar una a una las instrucciones del programa principal.

Begin

```
writeln('Ingrese la longitud de la secuencia: ');
```

```
readln(CANT);
```

```
ENCONTRE:=FALSE;
```

```
writeln('Ingrese la secuencia');
```

```
while ((CANT>0) and (ENCONTRE=FALSE)) do
```

```
begin
```

```
    read(Elem);
```

```
    CANT:=CANT-1;
```

```
    if codificar(elem)='A'
```

```
        then ENCONTRE:=TRUE;
```

```
end;
```

```
writeln('El primer elem. q codificado es una A es: ', Elem);
```

```
End.
```

Ingrese la longitud de la secuencia: 5
Ingrese la secuencia: 258 12 324 5 776

BUFFER

12

324

5

776

Memoria del programa principal

ELEM	258
CANT	5 4
ENCONTRE	FALSE

Comenzamos a ejecutar una a una las instrucciones del programa principal.

Begin

```
writeln('Ingrese la longitud de la secuencia: ');
```

```
readln(CANT);
```

```
ENCONTRE:=FALSE;
```

```
writeln('Ingrese la secuencia');
```

```
while ((CANT>0) and (ENCONTRE=FALSE)) do
```

```
begin
```

```
    read(Elem);
```

```
    CANT:=CANT-1;
```

```
    if codificar(elem)='A'
```

```
        then ENCONTRE:=TRUE;
```

```
end;
```

```
writeln('El primer elem. q codificado es una A es: ', Elem);
```

```
End.
```

Ingrese la longitud de la secuencia: 5
Ingrese la secuencia: 258 12 324 5 776

BUFFER

12

324

5

776

Memoria del programa principal

ELEM	258
CANT	5 4
ENCONTRE	FALSE

Comenzamos a ejecutar una a una las instrucciones del programa principal.

Begin

writeln('Ingrese la longitud de la secuencia: ');

readln(CANT);

ENCONTRE:=FALSE;

writeln('Ingrese la secuencia');

while ((CANT>0) and (ENCONTRE=FALSE)) do

begin

 read(Elem);

 CANT:=CANT-1;

 if **codificar(elem)='A'**
 then ENCONTRE:=TRUE;

end;

writeln('El primer elem. q codificado es una A es: ', Elem);

End.

Ingrese la longitud de la secuencia: 5
Ingrese la secuencia: 258 12 324 5 776

BUFFER

12

324

5

776

Memoria del programa principal

ELEM	258
CANT	5 4
ENCONTRE	FALSE

Memoria de la función (Registro de activación)

a	258
ult	
<i>codificar</i>	

Begin

writeln('Ingrese la longitud

readln(CANT);

ENCONTRE:=FALSE;

writeln('Ingrese la secuencia

while ((CANT>0) and (ENCONTRE=FALSE)) do

begin

read(Elem);

CANT:=CANT-1;

if **codificar(elem)='A'**

then ENCONTRE:=TRUE;

end;

writeln('El primer elem. q codificado es una A es: ', Elem);

End.

```
function codificar(a: integer): char;
var ult: integer;
begin
  ult:= a mod 10;
  case ult of
    0,1,2: codificar:='A';
    3,4,5: codificar:='E';
    6: codificar:='I';
    7,8: codificar:='O';
    9: codificar:='U';
  end;
end;
```

Ingrese la longitud de la secuencia: 5
Ingrese la secuencia: 258 12 324 5 776

BUFFER

12

324

5

776

Memoria de la función

- Al realizar una invocación a una primitiva, se prepara la memoria para ejecutar esa primitiva.
- El registro de activación (RA) de la función contiene un lugar para cada parámetro por valor, un lugar para cada variable local y un lugar especial para el resultado.
- Luego de ARMAR el RA, se procede a realizar el pasaje de parámetros. En este caso hay un único parámetro por valor (a) y a este se COPIA el valor del parámetro REAL (el valor de quien aparece en la llamada, en este caso ELEM que vale 258).
- Recién ahora podemos empezar a ejecutar una a una las instrucciones de la función.

Memoria del programa principal

ELEM	258
CANT	5 4
ENCONTRE	FALSE

Memoria de la función (Registro de activación)

a	258
ult	8
<i>codificar</i>	

Begin

writeln('Ingre

readln(CANT)

ENCONTRE:=I

writeln('Ingre

while ((CANT:

begin

```
function codificar(a: integer): char;
var ult: integer;
begin
    ult := a mod 10;
    case ult of
        0,1,2: codificar := 'A';
        3,4,5: codificar := 'E';
        6: codificar := 'I';
        7,8: codificar := 'O';
        9: codificar := 'U';
    end;
end;
```

read(Elem);

CANT:=CANT-1;

if **codificar(elem)='A'**
then ENCONTRE:=TRUE;

end;

writeln('El primer elem. q codificado es una A es: ', Elem);

End.

Ingrese la longitud de la secuencia: 5
Ingrese la secuencia: 258 12 324 5 776

BUFFER

12

324

5

776

Memoria del programa principal

ELEM	258
CANT	5 4
ENCONTRE	FALSE

Memoria de la función (Registro de activación)

a	258
ult	8
<i>codificar</i>	'O'

Begin

writeln('Ingre

readln(CANT)

ENCONTRE:=I

writeln('Ingre

while ((CANT:

begin

```
function codificar(a: integer): char;
var ult: integer;
begin
  ult:= a mod 10;
  case ult of
    0,1,2: codificar:='A';
    3,4,5: codificar:='E';
    6: codificar:='I';
    7,8: codificar:='O';
    9: codificar:='U';
  end;
end;
```

read(Elem);

CANT:=CANT-1;

if **codificar(elem)='A'**

then ENCONTRE:=TRUE;

end;

writeln('El primer elem. q codificado es una A es: ', Elem);

End.

Ingrese la longitud de la secuencia: 5
Ingrese la secuencia: 258 12 324 5 776

BUFFER

12

324

5

776

Retorno de la función

- Una vez que la función termina (llegamos al end de su bloque ejecutable) la ejecución vuelve a donde quedó pendiente en el programa principal.
- El valor que se obtuvo como resultado de la función se “reemplaza” temporalmente en la expresión donde está la invocación para completar la evaluación de la expresión.
- El RA de la función se destruye y se libera esa memoria.

Memoria del programa principal

ELEM	258
CANT	5 4
ENCONTRE	FALSE

Memoria de la función (Registro de activación)

a	258
ult	8
<i>codificar</i>	'0'

```

Begin
writeln('Ingrese la longitud de la secuencia: ');
readln(CANT);
ENCONTRE:=FALSE;
writeln('Ingrese la secuencia');
while ((CANT>0) and (ENCONTRE=FALSE)) do
begin
    read(Elem);
    CANT:=CANT-1;
    if 'O'='A' ..... DA FALSO
        then ENCONTRE:=TRUE;
end;
writeln('El primer elem. q codificado es una A es: ', Elem);
End.
    
```

Ingrese la longitud de la secuencia: 5
 Ingrese la secuencia: 258 12 324 5 776

BUFFER

	12	324	5	776
--	----	-----	---	-----

Memoria del programa principal

ELEM	258
CANT	5 4
ENCONTRE	FALSE

Memoria de la función (Registro de activación)

a	258
ult	8
<i>codificar</i>	'0'

Begin

writeln('Ingrese la longitud de la secuencia: ');

readln(CANT);

ENCONTRE:=FALSE;

writeln('Ingrese la secuencia');

while ((CANT>0) and (ENCONTRE=FALSE)) do

begin

read(Elem);

CANT:=CANT-1;

if codificar(elem)='A'

then ENCONTRE:=TRUE;

end;

writeln('El primer elem. q codificado es una A es: ', Elem);

End.

Las dos condiciones son VERDADERAS

Ingrese la longitud de la secuencia: 5
Ingrese la secuencia: 258 12 324 5 776

BUFFER

12

324

5

776

Memoria del programa principal

ELEM	12
CANT	5 4
ENCONTRE	FALSE

Memoria de la función (Registro de activación)

a	258
ult	8
<i>codificar</i>	'0'

```

Begin
writeln('Ingrese la longitud de la secuencia: ');
readln(CANT);
ENCONTRE:=FALSE;
writeln('Ingrese la secuencia');
while ((CANT>0) and (ENCONTRE=FALSE)) do
begin
    read(Elem);
    CANT:=CANT-1;
    if codificar(elem)='A'
        then ENCONTRE:=TRUE;
end;
writeln('El primer elem. q codificado es una A es: ', Elem);
End.
    
```

Ingrese la longitud de la secuencia: 5
 Ingrese la secuencia: 258 12 324 5 776

BUFFER

		324	5	776
--	--	-----	---	-----

Memoria del programa principal

ELEM	12
CANT	5 4 3
ENCONTRE	FALSE

Memoria de la función (Registro de activación)

a	258
ult	8
<i>codificar</i>	'0'

```

Begin
writeln('Ingrese la longitud de la secuencia: ');
readln(CANT);
ENCONTRE:=FALSE;
writeln('Ingrese la secuencia');
while ((CANT>0) and (ENCONTRE=FALSE)) do
begin
    read(Elem);
    CANT:=CANT-1;
    if codificar(elem)='A'
        then ENCONTRE:=TRUE;
end;
writeln('El primer elem. q codificado es una A es: ', Elem);
End.
    
```

Ingrese la longitud de la secuencia: 5
 Ingrese la secuencia: 258 12 324 5 776

BUFFER

		324	5	776
--	--	-----	---	-----

Memoria del programa principal

ELEM	12
CANT	5 4 3
ENCONTRE	FALSE

Memoria de la función (Registro de activación)

a	258
ult	8
<i>codificar</i>	'0'

Memoria de la función (Registro de activación)

a	12
ult	
<i>codificar</i>	

```

Begin
writeln('Ingrese la longitud de la secuencia: ');
readln(CANT);
ENCONTRE:=FALSE;
writeln('Ingrese la secuencia');
while ((CANT>0) and (ENCONTRE=FALSE)) do
begin
  read(Elem);
  CANT:=CANT-1;
  if codificar(elem)='A'
  then ENCONTRE:=TRUE;
end;
writeln('El primer elem. q codificado es una A es: ', Elem);
End.
  
```

Ingrese la longitud de la secuencia: 5
 Ingrese la secuencia: 258 12 324 5 776

BUFFER

		324	5	776
--	--	-----	---	-----

Memoria del programa principal

ELEM	12
CANT	5 4 3
ENCONTRE	FALSE

Memoria de la función (Registro de activación)

a	258
ult	8
<i>codificar</i>	'O'

```
function codificar(a: integer): char;
var ult: integer;
begin
  ult := a mod 10;
  case ult of
    0,1,2: codificar := 'A';
    3,4,5: codificar := 'E';
    6: codificar := 'I';
    7,8: codificar := 'O';
    9: codificar := 'U';
  end;
end;
```

Begin
writeln(
readln(
ENCON
writeln(
while ((
begin

```
  read(Elem);
  CANT:=CANT-1;
  if codificar(elem)='A'
  then ENCONTRE:=TRUE;
```

end;
writeln('El primer elem. q codificado es una A es: ', Elem);
End.

Memoria de la función (Registro de activación)

a	12
ult	2
<i>codificar</i>	

Ingrese la longitud de la secuencia: 5
Ingrese la secuencia: 258 12 324 5 776

BUFFER

324

5

776

Memoria del programa principal

ELEM	12
CANT	5 4 3
ENCONTRE	FALSE

Memoria de la función (Registro de activación)

a	258
ult	8
<i>codificar</i>	'O'

```
function codificar(a: integer): char;
var ult: integer;
begin
  ult := a mod 10;
  case ult of
    0, 1, 2: codificar := 'A';
    3, 4, 5: codificar := 'E';
    6: codificar := 'I';
    7, 8: codificar := 'O';
    9: codificar := 'U';
  end;
end;
```

```
Begin
writeln(
readln(
ENCON
writeln(
while ((
begin
  read(Elem);
  CANT:=CANT-1;
  if codificar(elem)='A'
    then ENCONTRE:=TRUE;
end;
writeln('El primer elem. q codificado es una A es: ', Elem);
End.
```

Memoria de la función (Registro de activación)

a	12
ult	2
<i>codificar</i>	'A'

Ingrese la longitud de la secuencia: 5
Ingrese la secuencia: 258 12 324 5 776

BUFFER

324

5

776

Memoria del programa principal

ELEM	12
CANT	5 4 3
ENCONTRE	FALSE

Memoria de la función (Registro de activación)

a	258
ult	8
<i>codificar</i>	'O'

Memoria de la función (Registro de activación)

a	12
ult	2
<i>codificar</i>	'A'

```

Begin
writeln('Ingrese la longitud de la secuencia: ');
readln(CANT);
ENCONTRE:=FALSE;
writeln('Ingrese la secuencia');
while ((CANT>0) and (ENCONTRE=FALSE)) do
begin
    read(Elem);
    CANT:=CANT-1;
    if 'A'='A' ----- DA VERDADERO
        then ENCONTRE:=TRUE;
end;
writeln('El primer elem. q codificado es una A es: ', Elem);
End.
    
```

Ingrese la longitud de la secuencia: 5
 Ingrese la secuencia: 258 12 324 5 776

BUFFER

		324	5	776
--	--	-----	---	-----

Memoria del programa principal

ELEM	12
CANT	5 4 3
ENCONTRE	TRUE

Memoria de la función (Registro de activación)

a	258
ult	8
<i>codificar</i>	'O'

```

Begin
writeln('Ingrese la longitud de la secuencia: ');
readln(CANT);
ENCONTRE:=FALSE;
writeln('Ingrese la secuencia');
while ((CANT>0) and (ENCONTRE=FALSE)) do
begin
    read(Elem);
    CANT:=CANT-1;
    if codificar(elem)='A'
        then ENCONTRE:=TRUE;
end;
writeln('El primer elem. q codificado es una A es: ', Elem);
End.
```

Memoria de la función (Registro de activación)

a	12
ult	2
<i>codificar</i>	'A'

Ingrese la longitud de la secuencia: 5
 Ingrese la secuencia: 258 12 324 5 776

BUFFER

		324	5	776
--	--	-----	---	-----

Memoria del programa principal

ELEM	12
CANT	5 4 3
ENCONTRE	TRUE

Memoria de la función (Registro de activación)

a	258
ult	8
<i>codificar</i>	'O'

Memoria de la función (Registro de activación)

a	12
ult	2
<i>codificar</i>	'A'

```

Begin
writeln('Ingrese la longitud de la secuencia: ');
readln(CANT);
ENCONTRE:=FALSE;
writeln('Ingrese la secuencia');
while ((CANT>0) and (ENCONTRE=FALSE)) do
begin
    read(Elem);
    CANT:=CANT-1;
    if codificar(elem)='A'
        then ENCONTRE:=TRUE;
end;
writeln('El primer elem. q codificado es una A es: ', Elem);
End.
    
```

La 2da cond. es FALSA, luego el AND es FALSO

Ingrese la longitud de la secuencia: 5
 Ingrese la secuencia: 258 12 324 5 776

BUFFER

		324	5	776
--	--	-----	---	-----

Memoria del programa principal

ELEM	12
CANT	5 4 3
ENCONTRE	TRUE

Memoria de la función (Registro de activación)

a	258
ult	8
<i>codificar</i>	'O'

```

Begin
writeln('Ingrese la longitud de la secuencia: ');
readln(CANT);
ENCONTRE:=FALSE;
writeln('Ingrese la secuencia');
while ((CANT>0) and (ENCONTRE=FALSE)) do
begin
    read(Elem);
    CANT:=CANT-1;
    if codificar(elem)='A'
        then ENCONTRE:=TRUE;
end;
writeln('El primer elem. q codificado es una A es: ', Elem);
End.
    
```

Memoria de la función (Registro de activación)

a	12
ult	2
<i>codificar</i>	'A'

Ingrese la longitud de la secuencia: 5
 Ingrese la secuencia: 258 12 324 5 776
 El primer elem. que codificado es una A es: 12

BUFFER

		324	5	776
--	--	-----	---	-----

Memoria del programa principal

ELEM	12
CANT	5 4 3
ENCONTRE	TRUE

Memoria de la función (Registro de activación)

a	258
ult	8
<i>codificar</i>	'O'

```

Begin
writeln('Ingrese la longitud de la secuencia: ');
readln(CANT);
ENCONTRE:=FALSE;
writeln('Ingrese la secuencia');
while ((CANT>0) and (ENCONTRE=FALSE)) do
begin
    read(Elem);
    CANT:=CANT-1;
    if codificar(elem)='A'
        then ENCONTRE:=TRUE;
end;
writeln('El primer elem. q codificado es una A es: ', Elem);

```

End.

Memoria de la función (Registro de activación)

a	12
ult	2
<i>codificar</i>	'A'

Ingrese la longitud de la secuencia: 5
 Ingrese la secuencia: 258 12 324 5 776
 El primer elem. que codificado es una A es: 12

BUFFER

		324	5	776
--	--	-----	---	-----

Memoria del programa principal

ELEM	12
CANT	5 4 3
ENCONTRE	TRUE

FOTO FINAL DE LA TRAZA

Tiene que incluir la memoria del programa principal, tachando pero dejando ver los valores anteriores, y los RA de las funciones que se fueron ejecutando.

Deberían estar uno debajo del otro!

Memoria de la función (Registro de activación)

a	258
ult	8
<i>codificar</i>	'O'

Memoria de la función (Registro de activación)

a	12
ult	2
<i>codificar</i>	'A'

Ingrese la longitud de la secuencia: 5

Ingrese la secuencia: 258 12 324 5 776

El primer elem. que codificado es una A es: 12

BUFFER

		324	5	776
--	--	-----	---	-----

ELEM	12
CANT	5 4 3
ENCONTRE	TRUE

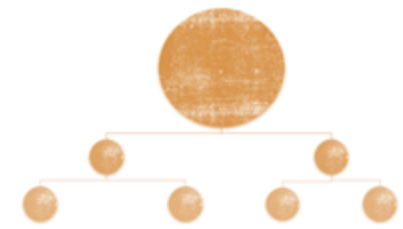
a	258
ult	8
<i>codificar</i>	'O'

a	12
ult	2
<i>codificar</i>	'A'

EJEMPLO con Trazo en el pizarrón

PROCEDIMIENTO





Problema: Queremos procesar una secuencia de caracteres, calculando la suma de todos los q correspondan a dígitos; la secuencia termina con un asterisco.

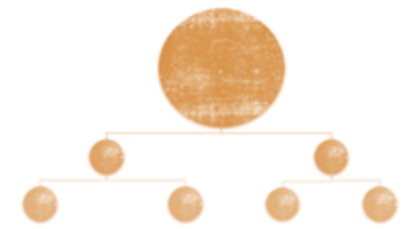
- Por ejemplo:

A24sbh.3 9 saz4*

Vamos a sumar: $2+4+3+9+4$

Devolvemos el resultado: 22



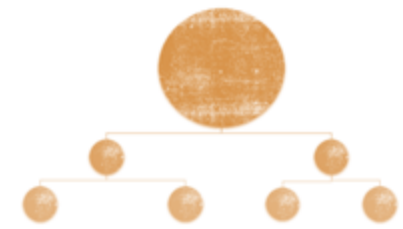


Problema: Queremos procesar una secuencia de caracteres, calculando la suma de todos los q correspondan a dígitos; la secuencia termina con un asterisco.

Problema principal: Recorrer completa una secuencia de caracteres con terminador (*).

Sub-problema: Dado un carácter, verificar si es dígito y en ese caso, convertirlo a número entero.





Procesar la secuencia de caracteres elemento a elemento hasta llegar al terminador.

Por cada caracter,

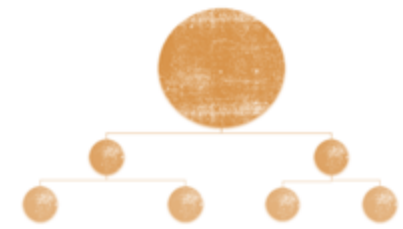
Si no es el terminador

vemos si corresponde a un dígito ('0' hasta '9'),

lo convertimos a dígito (tipo INTEGER),

lo acumulamos en un acumulador de sumas.



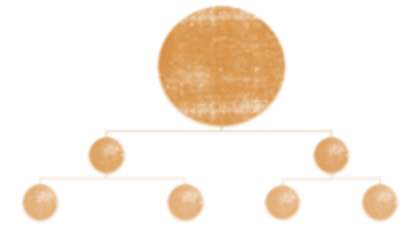


- Vamos a hacer un **procedimiento** que tome un caracter, se fije si corresponde a un dígito, y si es así lo convierta a entero.
- Nuestro procedimiento tiene un dato de entrada: el char
- Nuestro procedimiento tiene DOS DATOS DE SALIDA:
 - I dato LOGICO que indica si el character era o no un dígito
 - I dato ENTERO que es el valor del dígito (cuando esto sea possible). Si no era un dígito este valor será -1



TABLA DE CARACTE

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72
73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96
97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x



Procedure aDIGITO(ch: char; var sePUDO: Boolean; var d: integer);

Begin

If (ch>='0') and (ch<='9')

Then

Begin

sePUDO:=TRUE;

d:=ord(ch)-48;

End

Else

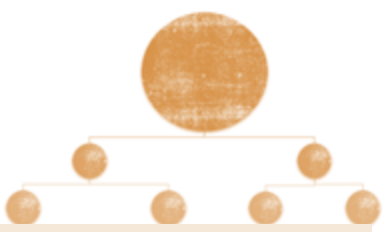
Begin

sePUDO:=FALSE;

d:=-1; // Cualquier valor porq no se debería usar

end

End;



Ord: recibe un dato de tipo char y retorna su posición en la tabla ascii (un entero)

Char → Ord → Integer



Ord('+') → 43

Ord('r') → 114

Ord('0') → 48

Ord('9') → 57



Program secuenciaSumoDigitos;

Var caracter: char; digito, suma: integer; esDigito: Boolean;

procedure aDIGITO(ch: char; var sePUDO: Boolean; var d: integer);

...

begin

Write('Ingrese la secuencia de caracteres'); suma:=0;

Repeat

 Read(caracter);

 if (caracter <> '*') then

 begin

 aDIGITO(caracter, esDigito, digito);

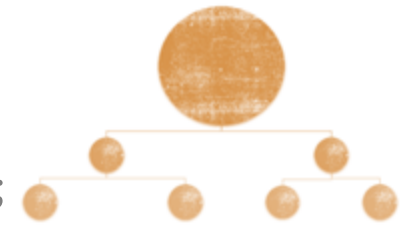
 if (esDigito=TRUE) then suma:=suma + digito;

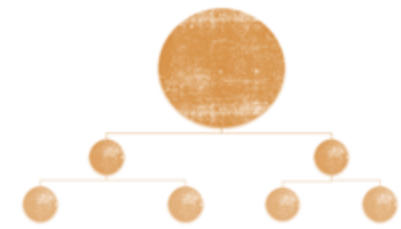
 End;

Until (caracter='*');

Write('La suma de los digitos q aparecieron en el texto es:', suma);

end.





Traza en el pizarrón

31 lines compiled, 0.0 sec

Ingrese la secuencia de caracteres: Hay 12 perros, 5 gatos y 25 conejos.*

La suma de los digitos q aparecieron en el texto es: 15

31 lines compiled, 0.0 sec

Ingrese la secuencia de caracteres: Sin animales.*

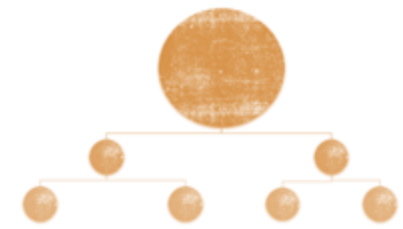
La suma de los digitos q aparecieron en el texto es: 0

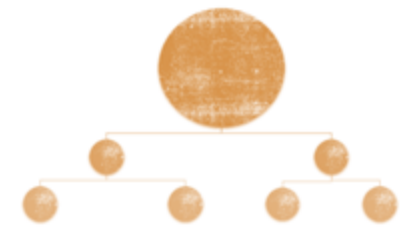


```

1 Program secuenciaSumoDigitos;
2 Var caracter: char; digito, suma: integer; esDigito: Boolean;
3
4 Procedure aDIGITO(ch: char; var sePUDO: Boolean; var d: integer);
5 Begin
6   If (ch>='0') and (ch<='9')
7   Then
8     Begin
9       sePUDO:=TRUE;
10      d:=ord(ch)-48;
11    End
12  Else
13    Begin
14      sePUDO:=FALSE;
15      d:=-1; // Cualquier valor porq no se debería usar
16    end
17  End;
18
19 begin
20   Write('Ingrese la secuencia de caracteres: ');
21   suma:=0;
22   Repeat
23     Read(caracter);
24     if (caracter <> '*') then
25     begin
26       aDIGITO(caracter, esDigito, digito);
27       if (esDigito=TRUE) then suma:=suma + digito;
28     End;
29   Until (caracter='*');
30   Write('La suma de los digitos q aparecieron en el texto es: ', suma);
31 end.

```



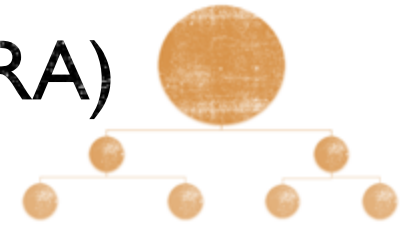


TIPs Traza

- Llamada a primitiva
 - Se arma la estructura del RA de la primitiva con tantos lugares como parámetros formales y variables locales tenga. Si es una función también se agrega un lugar especial para la función.
 - Se realiza el pasaje de parámetros:
 - Si es parámetro por valor, se copia en el parámetro formal el valor q se encuentra en el lugar correspondiente de parámetro real.
 - Si es parámetro por variable, se coloca una referencia (flecha) al parámetro actual correspondiente, siempre mirando LA INVOCACION.
 - Se procede a ejecutar la primitiva hasta el end de su bloque ejecutable.
 - Si es función, se copia el resultado (almacenado en el nombre de la función) en reemplazo de la invocación que dio lugar a la ejecución actual de la primitiva.



La estructura del Registro de Activación (RA) del procedimiento será siempre la misma:



ch	
sePudo	⊙ →
d	⊙ →

“ch” es parámetro **POR VALOR**, en su celda se colocará el valor del parámetro **REAL** que le corresponde según la invocación

“sePudo” y “d” son parámetros **POR VARIABLE**, en su celda se colocarán **FLECHAS** apuntando a la celda del parámetro **REAL** correspondiente según la invocación.



LA FORMA DEL RA de una misma Primitiva es siempre igual.

elem	'a' '2' '1' '*'
digito	2 1
suma	0 2 3
esDigito	F T T

Porción de memoria del P. PRAL

consola

221*

RA aDigito
aDigito ('a', m)

RA aDigito
aDigito ('2', m)

buffer

'a' '2' '1' '*'

Manejo de la memoria



Cuando comienza la ejecución de un programa en Pascal se reservan una o más celdas en memoria para cada una de las variables globales (**variables declaradas en el programa principal**).

Registro
de activación

Cuando un procedimiento o función se **invoca** se reservan celdas para cada una de las variables locales al procedimiento o función, como así también para los parámetros.

Para cada parámetro pasado por referencia se reserva una celda que mantiene la referencia (flecha) al parámetro real.

Todas estas celdas son liberadas cuando el procedimiento o función termina.



Manejo de la memoria

Program parametros;
var pa1, pa2: integer;

Variables globales

```
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1, pf2, local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1, pf2, local);  
end;
```

Parámetros formales

¿COMO ES
LA ESTRUCTURA
DEL RA DE P?

```
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1, pa2);  
  p(pa1, pa2);  
  writeln(pa1, pa2);  
end.
```

Parámetros reales

Definición: Variable global es aquella variable declarada en el PROGRAMA PRINCIPAL.

Manejo de la memoria

Flujo de ejecución

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    local := pf1 + pf2;  
    writeln(pf1, pf2, local);  
    pf1 := pf1 + 1;    pf2 := local + 1;  
    writeln(pf1, pf2, local);  
end;  
begin  
    pa1 := 1; pa2 := 3;  
    writeln(pa1, pa2);  
    p(pa1, pa2);  
    writeln(pa1, pa2);  
end.
```

Memoria

Consola



Manejo de la memoria

Flujo de ejecución

Program parametros;

var pa1, pa2: integer;

procedure p(pf1: integer; var pf2: integer);

var local: integer;

begin

local := pf1 + pf2;

writeln(pf1, pf2, local);

pf1 := pf1 + 1; pf2 := local + 1;

writeln(pf1, pf2, local);

end;

begin

→ pa1:=1; pa2:=3;

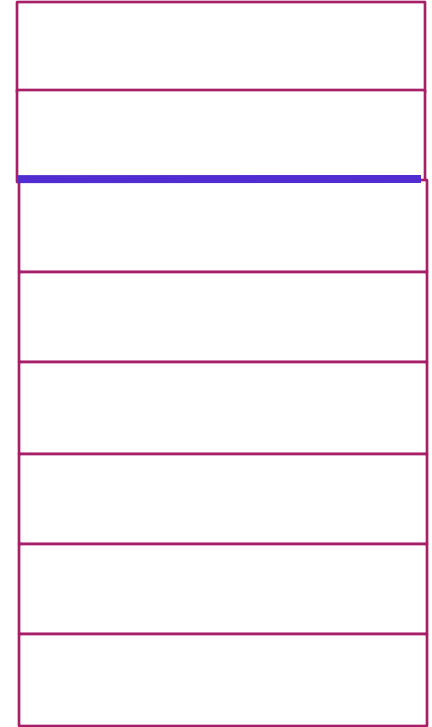
writeln(pa1, pa2);

p(pa1, pa2);

writeln(pa1, pa2);

end.

Memoria



Consola



Comienza la ejecución del programa

Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    local := pf1 + pf2;  
    writeln(pf1, pf2, local);  
    pf1 := pf1 + 1; pf2 := local + 1;  
    writeln(pf1, pf2, local);  
end;  
begin  
    pa1 := 1; pa2 := 3;  
    writeln(pa1, pa2);  
    p(pa1, pa2);  
    writeln(pa1, pa2);  
end.
```

Flujo de ejecución

Memoria

pa1
pa2

1

Consola



Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    local := pf1 + pf2;  
    writeln(pf1, pf2, local);  
    pf1 := pf1 + 1; pf2 := local + 1;  
    writeln(pf1, pf2, local);  
end;  
begin  
    pa1 := 1; pa2 := 3;  
    writeln(pa1, pa2);  
    p(pa1, pa2);  
    writeln(pa1, pa2);  
end.
```

Flujo de ejecución

Memoria

pa1
pa2

1
3

Consola



Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    local:=pf1+pf2;  
    writeln(pf1,pf2,local);  
    pf1:= pf1 + 1; pf2:=local + 1;  
    writeln(pf1,pf2,local);  
end;  
begin  
    pa1:=1; pa2:=3;  
    writeln(pa1,pa2);  
    p(pa1,pa2);  
    writeln(pa1,pa2);  
end.
```

Flujo de ejecución

Memoria

pa1
pa2

1
3

Consola

1 3

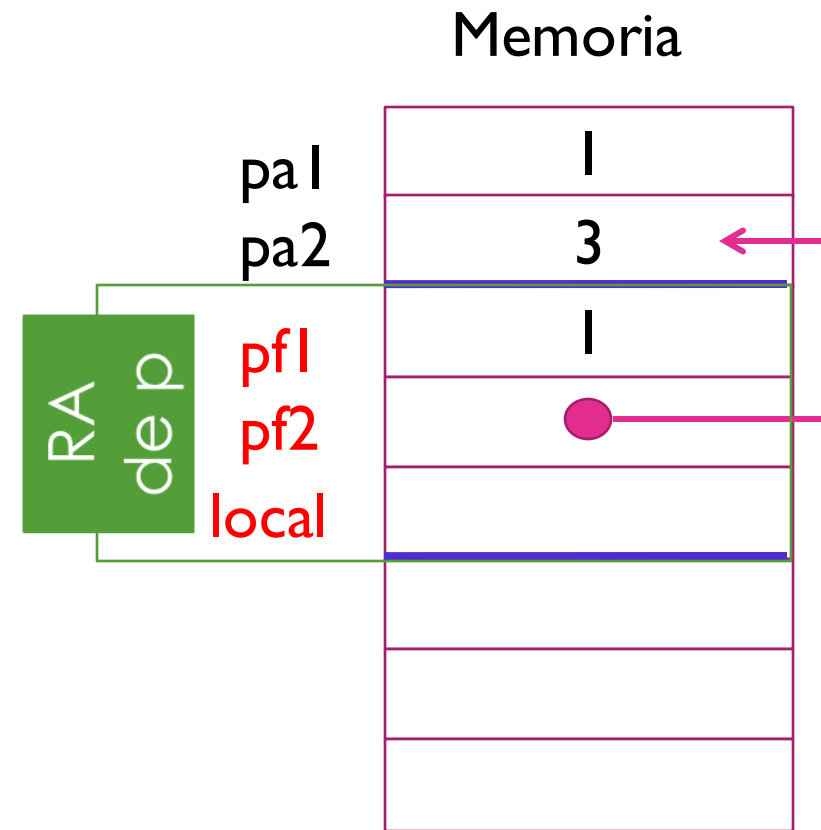
Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1,pf2,local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1,pf2,local);  
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1,pa2);  
→ p(pa1,pa2); { SUSPENDIDO }  
  writeln(pa1,pa2);  
end.
```

Invocación a p:

- Se crea su RA (Registro de Activación).
- El control pasa al procedimiento p.
- El bloque del programa principal se suspende.

Flujo de ejecución



Consola

1 3

ARMADO del Registro de activación de un procedimiento:

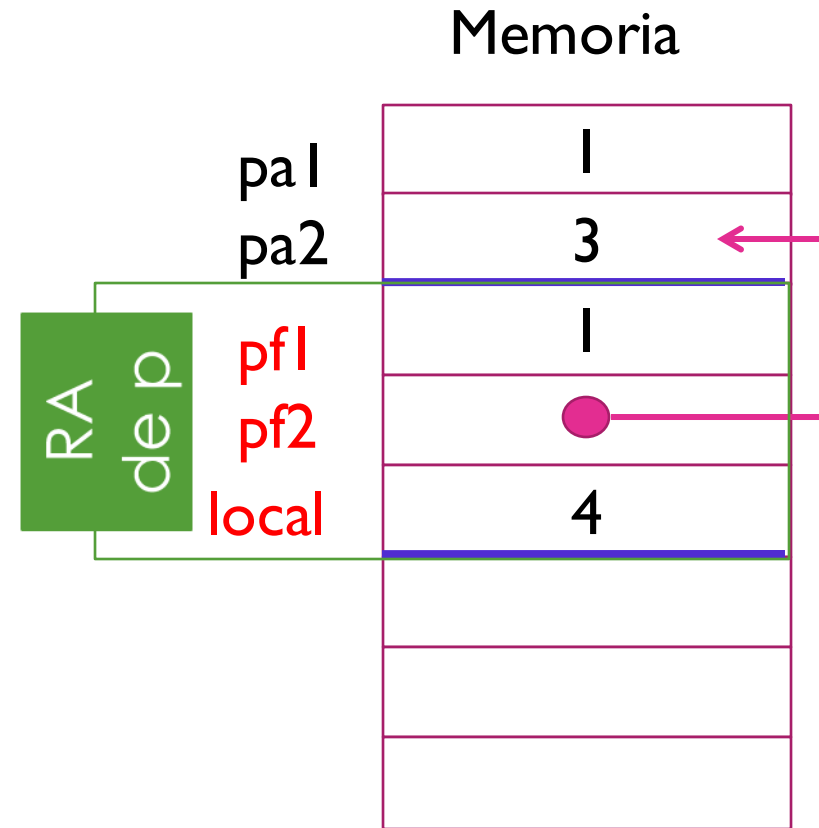
- Se guarda una celda para cada parámetro formal.
- Se guarda una celda para cada variable local.
- Pasaje de parámetros
 - En la celda correspondiente a cada parámetro formal POR VALOR se pone el valor del parámetro real asociado (buscar en la invocación).
 - En la celda correspondiente a cada parámetro formal POR REFERENCIA se pone una flecha a la dirección del parámetro real asociado (buscar en la invocación).

**SIEMPRE
IGUAL**

Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1,pf2,local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1,pf2,local);  
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1,pa2);  
  p(pa1,pa2); { SUSPENDIDO }  
  writeln(pa1,pa2);  
end.
```

Flujo de ejecución



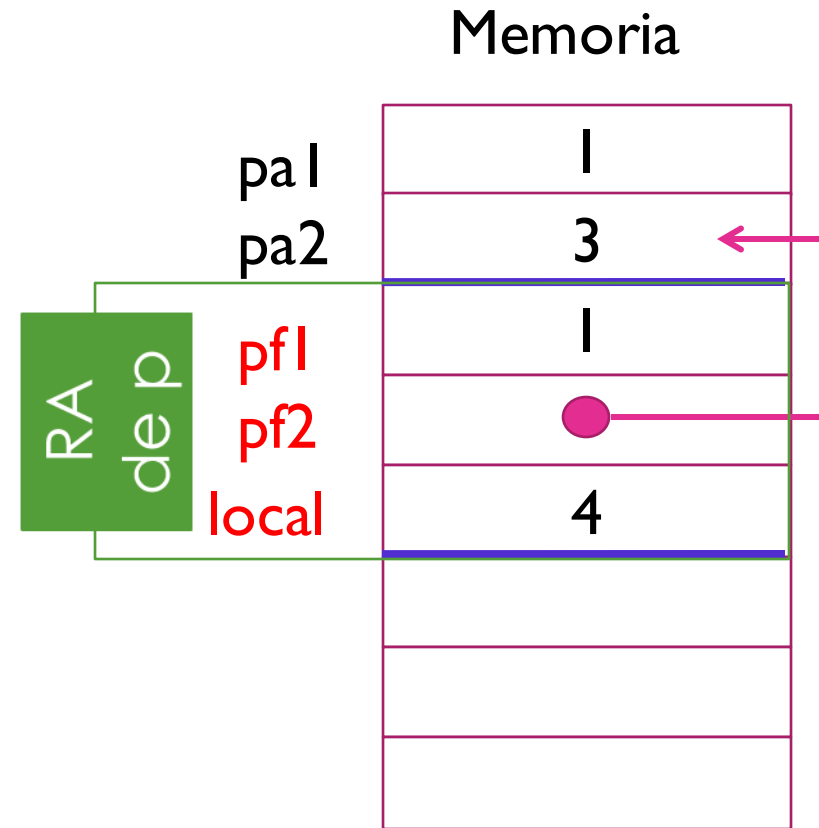
Consola

1 3

Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1,pf2,local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1,pf2,local);  
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1,pa2);  
  p(pa1,pa2); { SUSPENDIDO }  
  writeln(pa1,pa2);  
end.
```

Flujo de ejecución



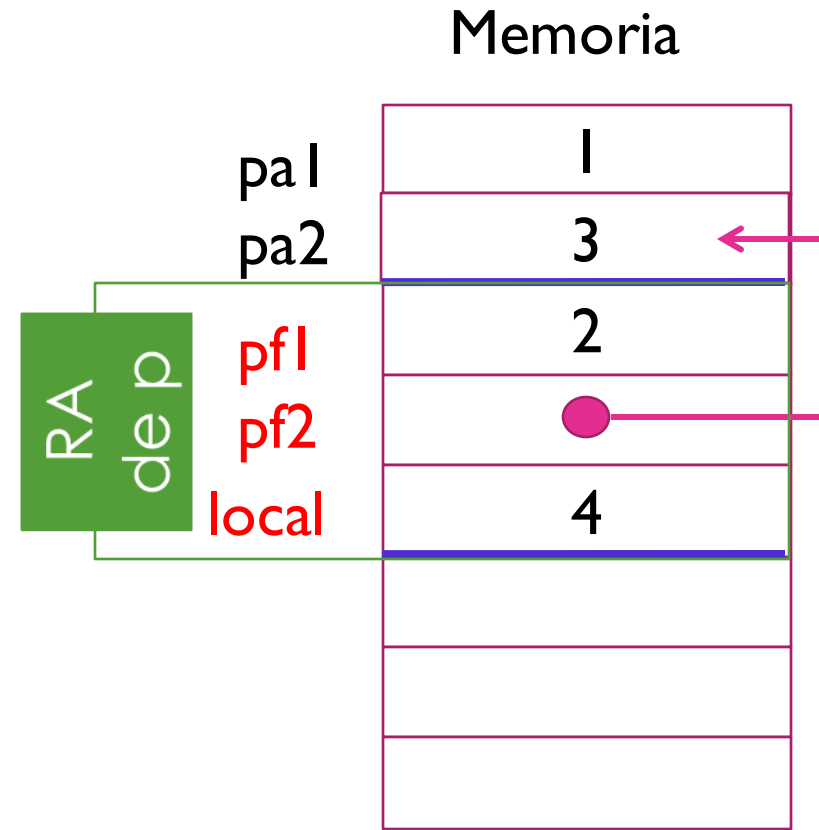
Consola

```
1 3  
1 3 4
```

Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1,pf2,local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1,pf2,local);  
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1,pa2);  
  p(pa1,pa2); { SUSPENDIDO }  
  writeln(pa1,pa2);  
end.
```

Flujo de ejecución



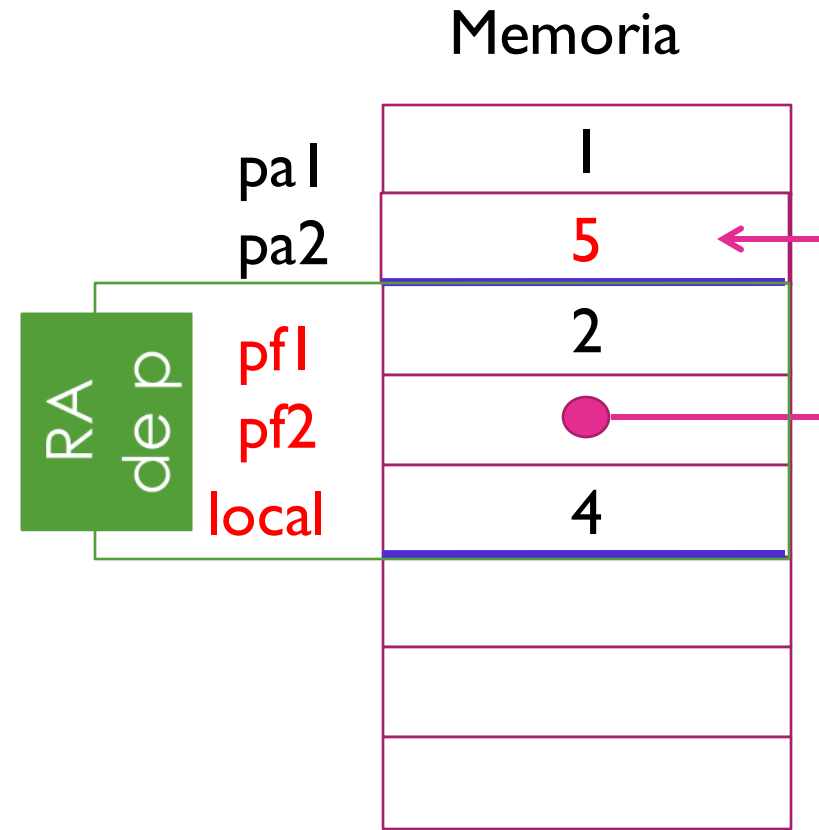
Consola

```
1 3  
1 3 4
```

Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1,pf2,local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1,pf2,local);  
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1,pa2);  
  p(pa1,pa2); { SUSPENDIDO }  
  writeln(pa1,pa2);  
end.
```

Flujo de ejecución



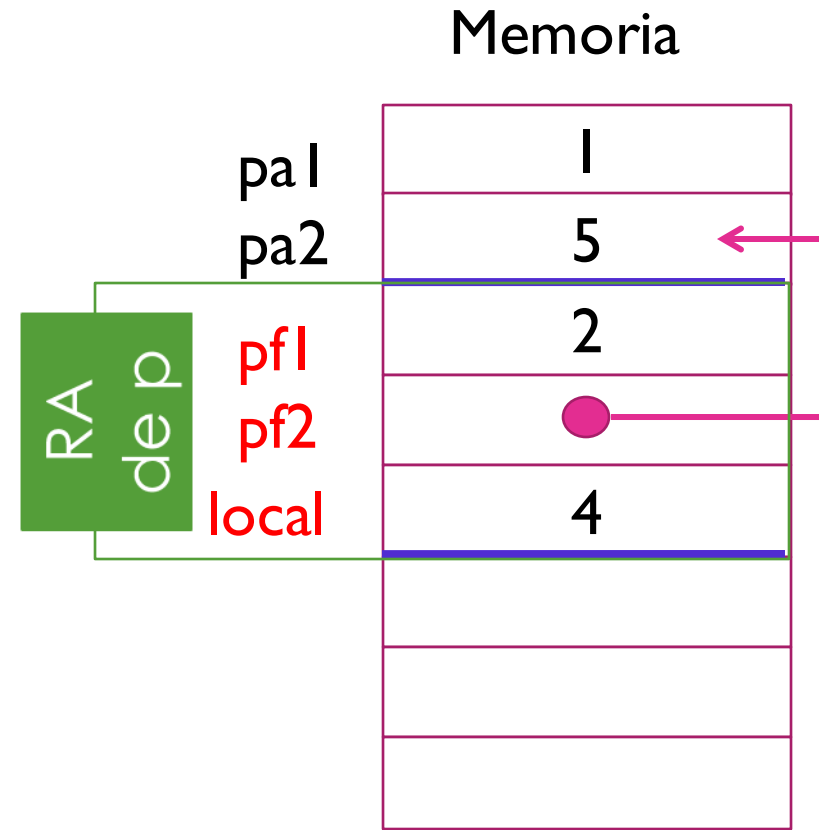
Consola

```
1 3  
1 3 4
```

Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1,pf2,local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1,pf2,local);  
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1,pa2);  
  p(pa1,pa2); { SUSPENDIDO }  
  writeln(pa1,pa2);  
end.
```

Flujo de ejecución



Consola

```
1 3  
1 3 4  
2 5 4
```

Manejo de la memoria

Flujo de ejecución

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin
```

```
local:=pf1;  
writeln('p: ', local);  
pf2:= pf1;  
writeln('p: ', pf2);
```

```
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1, pa2);  
  p(pa1, pa2);  
  → writeln(pa1, pa2);  
end.
```

El procedimiento termina, la memoria asignada al procedimiento se libera y la ejecución continúa en la instrucción siguiente a la instrucción de la invocación.

Memoria

pa1
pa2

1
5

Consola

```
1 3  
1 3 4  
2 5 4  
1 5
```

Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    local:=pf1+pf2;  
    writeln(pf1,pf2,local);  
    pf1:= pf1+1; pf2:=local+1;  
    writeln(pf1,pf2,local);  
end;  
begin  
    pa1:=1; pa2:=3;  
    writeln(pa1,pa2);  
    p(pa1,pa2);  
    writeln(pa1,pa2);  
end.
```

Flujo de ejecución

Memoria

pa1	1
pa2	5

Consola

```
1 3  
1 3 4  
2 5 4  
1 5
```


Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    local:=pf1+pf2;  
    writeln(pf1,pf2,local);  
    pf1:= pf1+1; pf2:=local+1;  
    writeln(pf1,pf2,local);  
end;  
begin  
    pa1:=1; pa2:=3;  
    writeln(pa1,pa2);  
    p(pa1,pa2);  
    writeln(pa1,pa2);  
    p(pa2, pa1); ←  
    writeln(pa1, pa2);  
end.
```

Se agrega
una
invocación a
p

Flujo de ejecución

Memoria

pa1	1
pa2	5

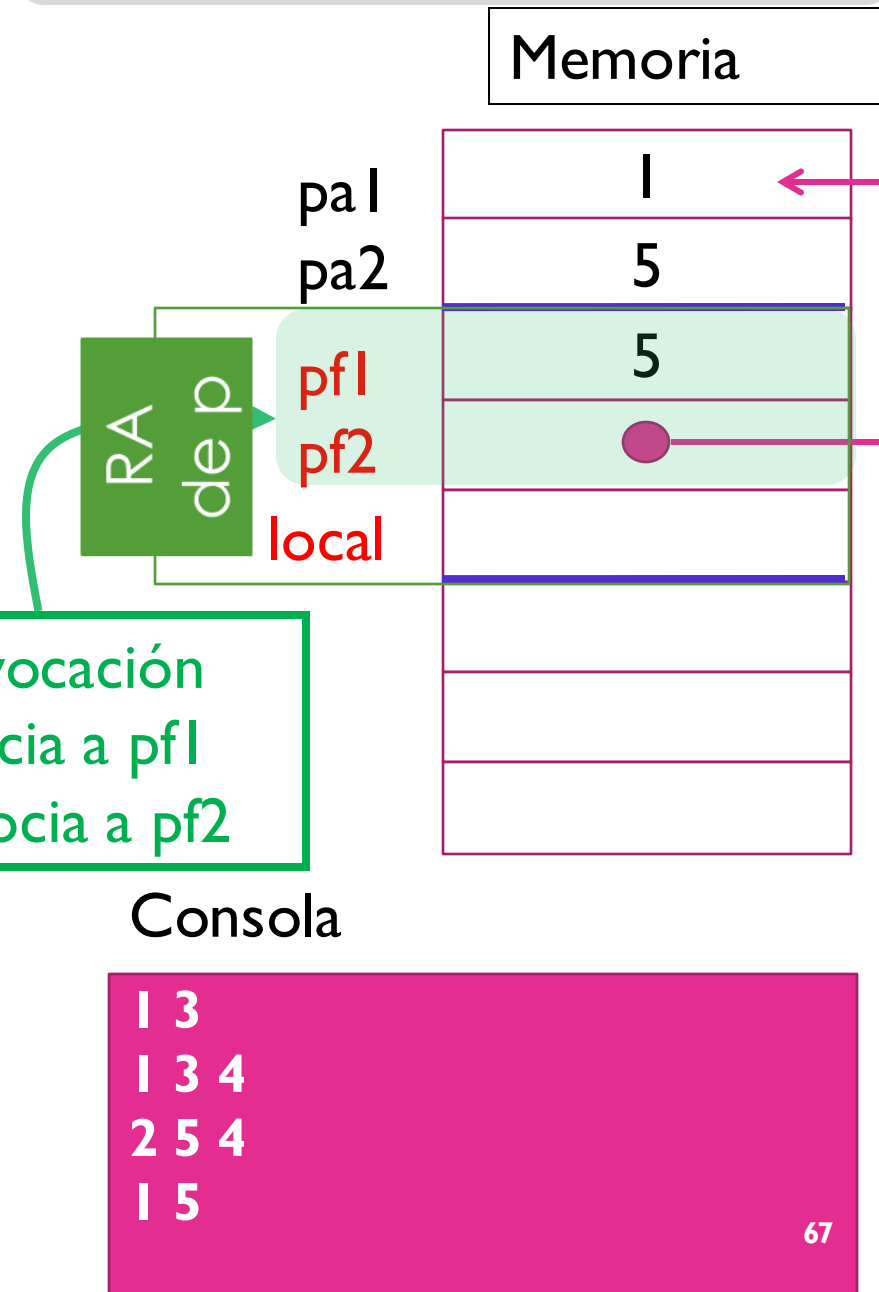
Consola

```
1 3  
1 3 4  
2 5 4  
1 5
```

Manejo de la memoria

Flujo de ejecución

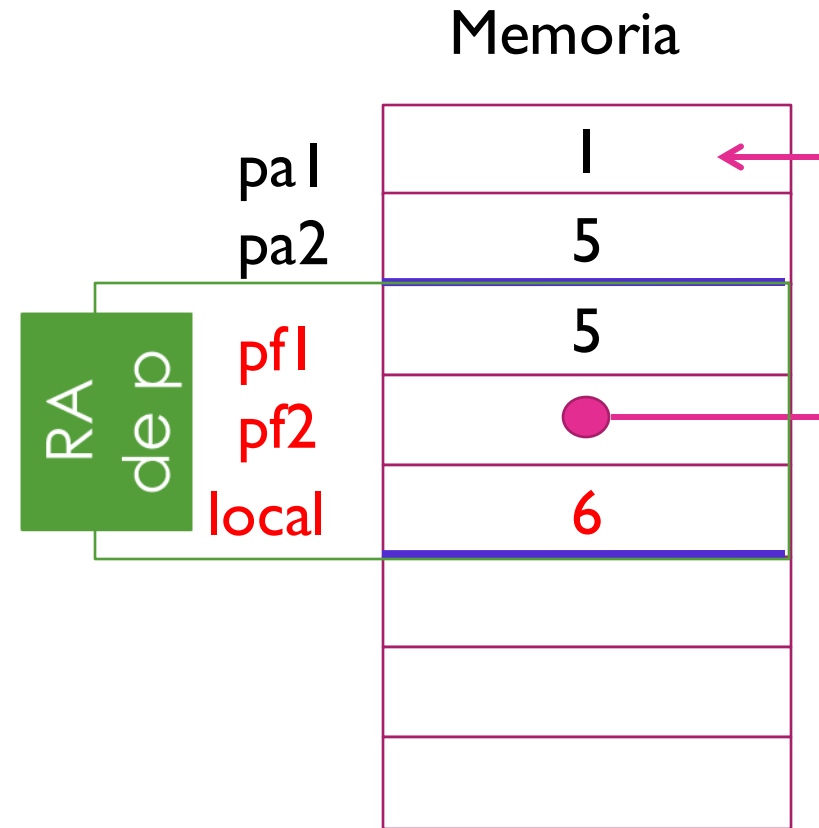
```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1,pf2,local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1,pf2,local);  
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1,pa2);  
  p(pa1,pa2);  
  writeln(pa1,pa2);  
  p(pa2, pa1);  
  writeln(pa1,pa2);  
end.
```



Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1,pf2,local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1,pf2,local);  
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1,pa2);  
  p(pa1,pa2);  
  writeln(pa1,pa2);  
  p(pa2, pa1); { SUSPENDIDO }  
  writeln(pa1,pa2);  
end.
```

Flujo de ejecución



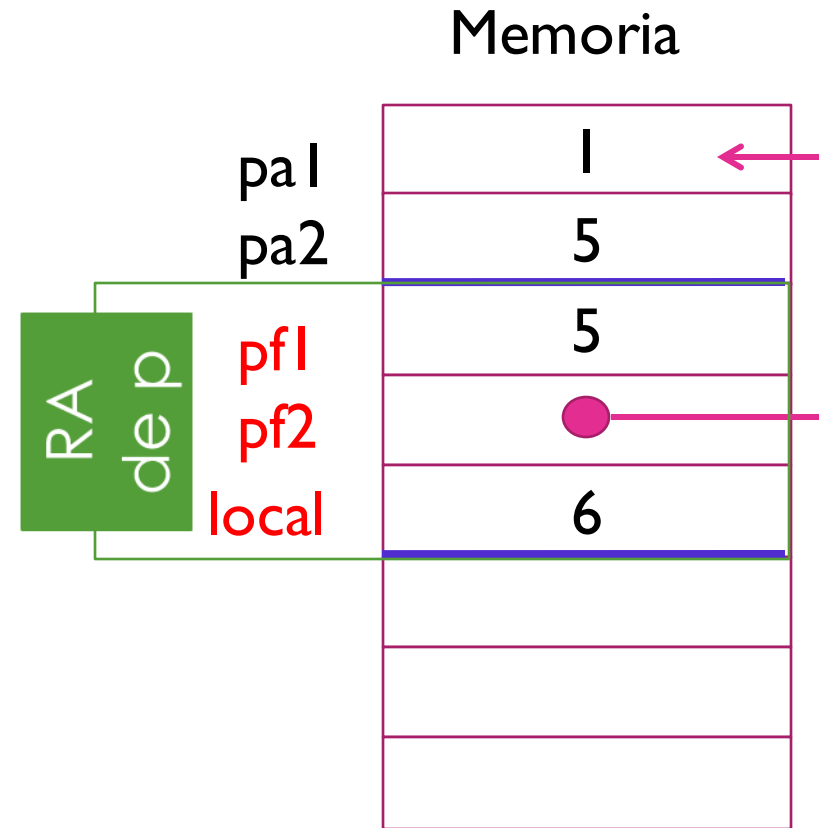
Consola

```
1 3  
1 3 4  
2 5 4  
1 5
```

Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1,pf2,local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1,pf2,local);  
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1,pa2);  
  p(pa1,pa2);  
  writeln(pa1,pa2);  
  p(pa2, pa1); { SUSPENDIDO }  
  writeln(pa1,pa2);  
end.
```

Flujo de ejecución



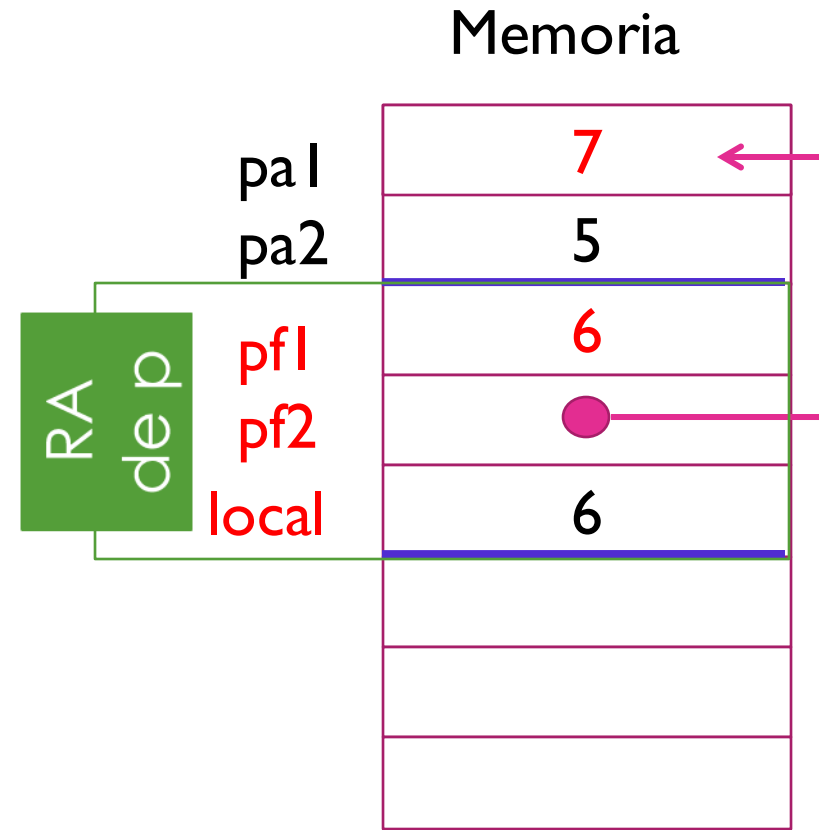
Consola

...
5 1 6

Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1,pf2,local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1,pf2,local);  
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1,pa2);  
  p(pa1,pa2);  
  writeln(pa1,pa2);  
  p(pa2, pa1); { SUSPENDIDO }  
  writeln(pa1,pa2);  
end.
```

Flujo de ejecución



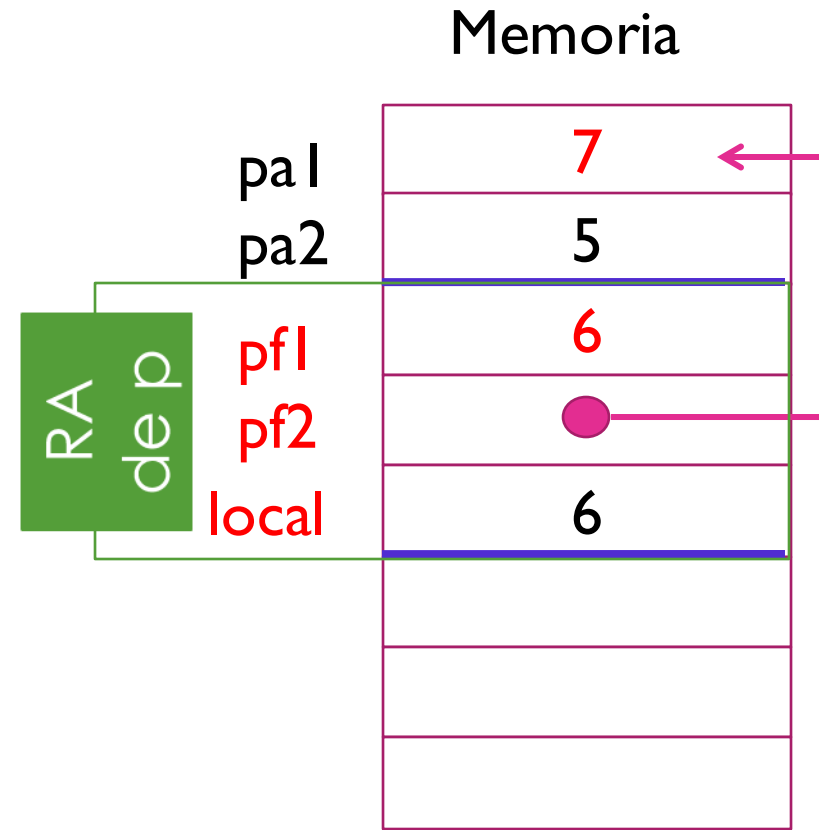
Consola

...
5 1 6

Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1,pf2,local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1,pf2,local);  
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1,pa2);  
  p(pa1,pa2);  
  writeln(pa1,pa2);  
  p(pa2, pa1); { SUSPENDIDO }  
  writeln(pa1,pa2);  
end.
```

Flujo de ejecución



Consola

```
...  
5 1 6  
6 7 6
```

Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    local:=pf1+pf2;  
    writeln(pf1,pf2,local);  
    pf1:= pf1+1; pf2:=local+1;  
    writeln(pf1,pf2,local);  
end;  
begin  
    pa1:=1; pa2:=3;  
    writeln(pa1,pa2);  
    p(pa1,pa2);  
    writeln(pa1,pa2);  
    p(pa2, pa1);  
    → writeln(pa1,pa2);  
end.
```

Flujo de ejecución

Memoria

pa1	7
pa2	5

Consola

```
...  
5 1 6  
6 7 6
```

Manejo de la memoria

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1,pf2,local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1,pf2,local);  
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1,pa2);  
  p(pa1,pa2);  
  writeln(pa1,pa2);  
  p(pa2, pa1);  
  writeln(pa1,pa2);  
end.
```

Flujo de ejecución

Memoria

pa1	7
pa2	5

Consola

```
...  
5 1 6  
6 7 6  
7 5
```


Manejo de la memoria

Flujo de ejecución

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    local:=pf1+pf2;  
    writeln(pf1,pf2,local);  
    pf1:= pf1+1; pf2:=local+1;  
    writeln(pf1,pf2,local);  
end;  
begin  
    pa1:=1; pa2:=3;  
    writeln(pa1,pa2);  
    p(pa1,pa2);  
    writeln(pa1,pa2);  
    p(pa2, pa1);  
    writeln(pa1,pa2);  
    p(-10,pa1);  
    writeln(pa1,pa2);  
end.
```



Memoria

pa1
pa2

7
5

Consola

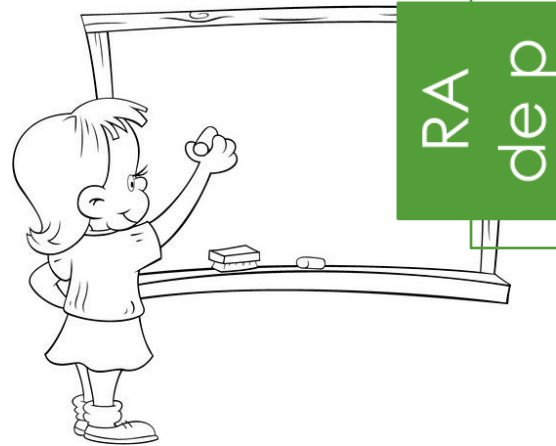
```
...  
7 5
```

Se agrega una
invocación a p

Manejo de la memoria

Flujo de ejecución

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1,pf2,local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1,pf2,local);  
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1,pa2);  
  p(pa1,pa2);  
  writeln(pa1,pa2);  
  p(pa2, pa1);  
  writeln(pa1,pa2);  
  p(-10,pa1);  
  writeln(pa1,pa2);  
end.
```



Memoria

pa1	7	←
pa2	5	
pf1	-10	
pf2		
local		

Consola

```
...  
5 1 6  
6 7 6  
7 5
```

Manejo de la memoria

Flujo de ejecución

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1,pf2,local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1,pf2,local);  
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1,pa2);  
  p(pa1,pa2);  
  writeln(pa1,pa2);  
  p(pa2, pa1);  
  writeln(pa1,pa2);  
  p(-10,pa1);  
  writeln(pa1,pa2);  
end.
```



Memoria

pa1	-2
pa2	5
pf1	-9
pf2	
local	-3

Consola

```
...  
-10 7 -3  
-9 -2 -3
```

Manejo de la memoria

Flujo de ejecución

```
Program parametros;  
var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local:=pf1+pf2;  
  writeln(pf1,pf2,local);  
  pf1:= pf1+1; pf2:=local+1;  
  writeln(pf1,pf2,local);  
end;  
begin  
  pa1:=1; pa2:=3;  
  writeln(pa1,pa2);  
  p(pa1,pa2);  
  writeln(pa1,pa2);  
  p(pa2, pa1);  
  writeln(pa1,pa2);  
  p(-10,pa1);  
  writeln(pa1,pa2);  
end.
```



Memoria

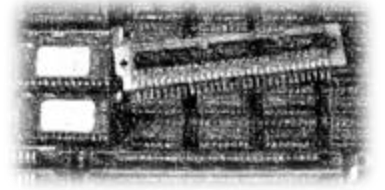
pa1	-2
pa2	5

Consola

```
...  
-10 7 -3  
-9 -2 -3  
-2 5
```

Manejo de memoria

Variables GLOBALES



```
program paramProc:
  Var pa1, pa2: integer;
  procedure p(pf1: integer; var pf2: integer);
  var local: integer;
  Begin
    local := pf1+pf2;
    pf1:= pf1+1; pf2 :=local+1;
    writeln(pf1:5,pf2:5,local:5);
  End;
  Begin
    p(5, -10);
    ...
  End.
```

```
error1
1  program error1;
.  var pa1,pa2:integer;
.  procedure p(pf1: integer; var pf2: integer);
.  var local:integer;
5  Begin
.    local := pf1+pf2;
.    pf1:= pf1+1; pf2 :=local+1;
.    writeln(pf1:5,pf2:5,local:5);
.  End;
10 Begin
11  p(pa1, -10);
.    writeln(pa1:5,pa2:5);
.  End;
14
```

Ventana de Mensajes

- Opciones cambiadas, recompila limpiando con -8
- error1.pas(11,14) Error: Variable identifier expected
- error1.pas(3,11) Hint: Found declaration: p(LongInt,var LongInt);
- error1.pas(12,15) Warning: Variable "pa1" does not seem to be initialized
- error1.pas(12,21) Warning: Variable "pa2" does not seem to be initialized
- error1.pas(15) Fatal

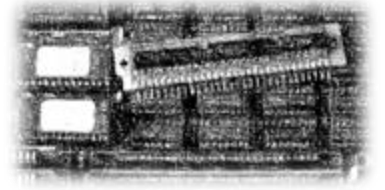
“ERROR: Se espera el nombre de una variable”



Error en compilación

El parámetro real no puede ser una constante cuando el pasaje de parámetros es por referencia

Manejo de memoria



```
program paramProc;  
Var pa1, pa2: integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
Begin  
    local := pf1+pf2;  
    pf1:= pf1+1; pf2 :=local+1;  
    writeln(pf1:5,pf2:5,local:5);  
End;  
Begin  
    p(sqr(5*10), pa2+1);  
    ..  
End.
```

```
1 program error1;  
  var pa1,pa2:integer;  
  procedure p(pf1: integer; var pf2: integer);  
  var local:integer;  
5 Begin  
  local := pf1+pf2;  
  pf1:= pf1+1; pf2 :=local+1;  
  writeln(pf1:5,pf2:5,local:5);  
  End;  
10 Begin  
11   p(pa1, pa2+1);  
  writeln(pa1:5,pa2:5);  
  End.  
14
```

Ventana de Mensajes

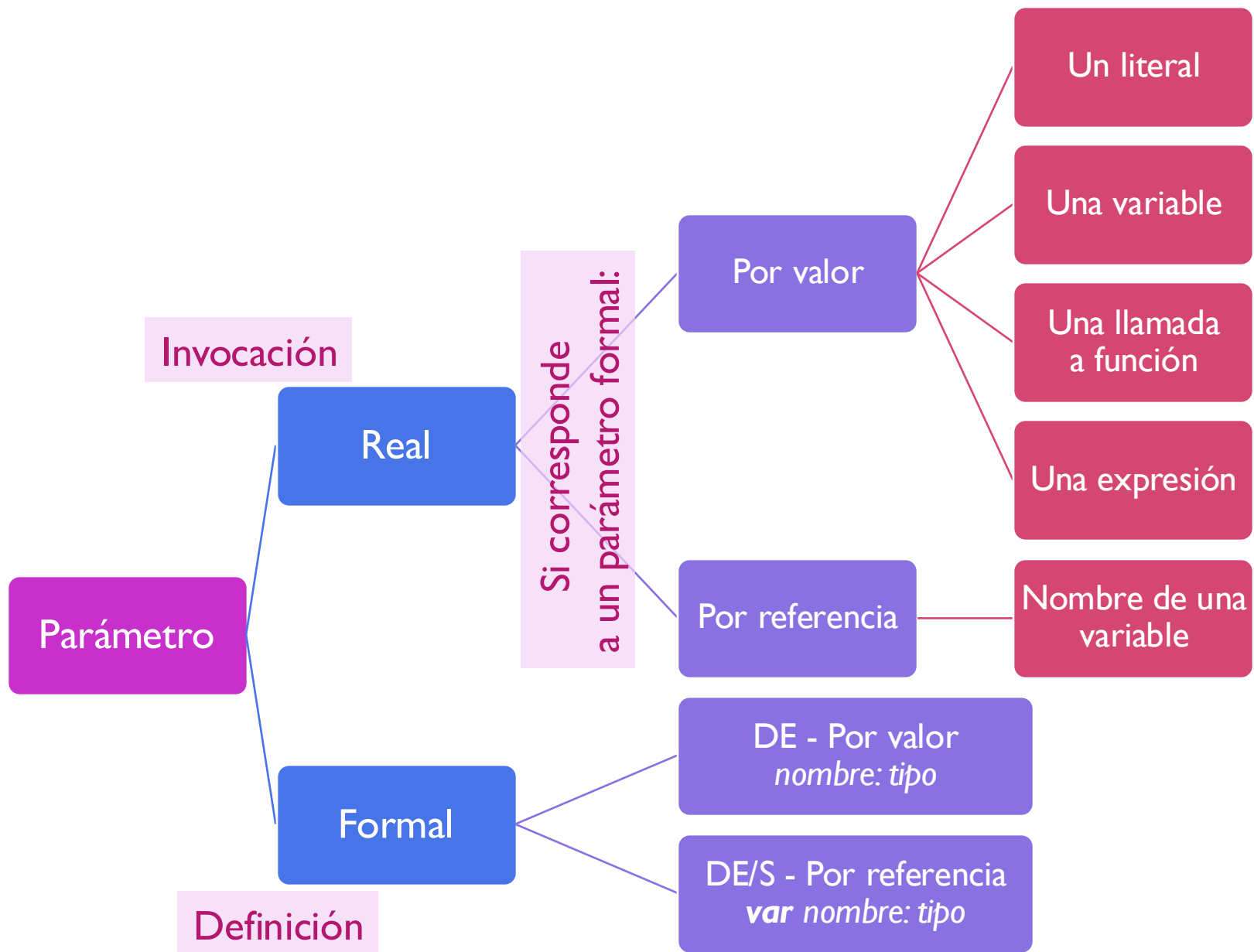
- error1.pas(11,11) Warning: Variable "pa2" does not seem to be initialized
- error1.pas(11,16) Error: Variable identifier expected
- error1.pas(3,11) Hint: Found declaration: p(LongInt,var LongInt);
- error1.pas(12,15) Warning: Vari
- error1.pas(15) Fatal: There wer

Error en compilación

“ERROR: Se espera el nombre
de una variable”



El parámetro real **no puede ser una expresión**
cuando el pasaje de parámetros es **por referencia**



Manejo de memoria



Dibujar la memoria al momento del **Begin** del bloque ejecutable de P.

```
Program MEMO;
```

```
Var a, b, c, d: integer;
```

```
Procedure P(x1: integer; var x2, x3: integer);
```

```
Var z: integer;
```

```
Begin
```

```
...
```

```
End;
```

```
Begin
```

```
a:=2; b:=5;
```

```
P(a+b, c, d);
```

```
...
```

```
End.
```



Ejercicio de coloquio



Manejo de memoria

Flujo de ejecución

```
program paramProc;  
var pa1, pa2:integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    local := pf1+pf2;  
    pf1:= pf1+10; pf2 :=local+1;  
    writeln(pf1, pf2, local);  
end;  
begin  
    pa1:=1; pa2:=5;  
    p(pa1, pa1);  
    writeln(pa1, pa2);  
end.
```

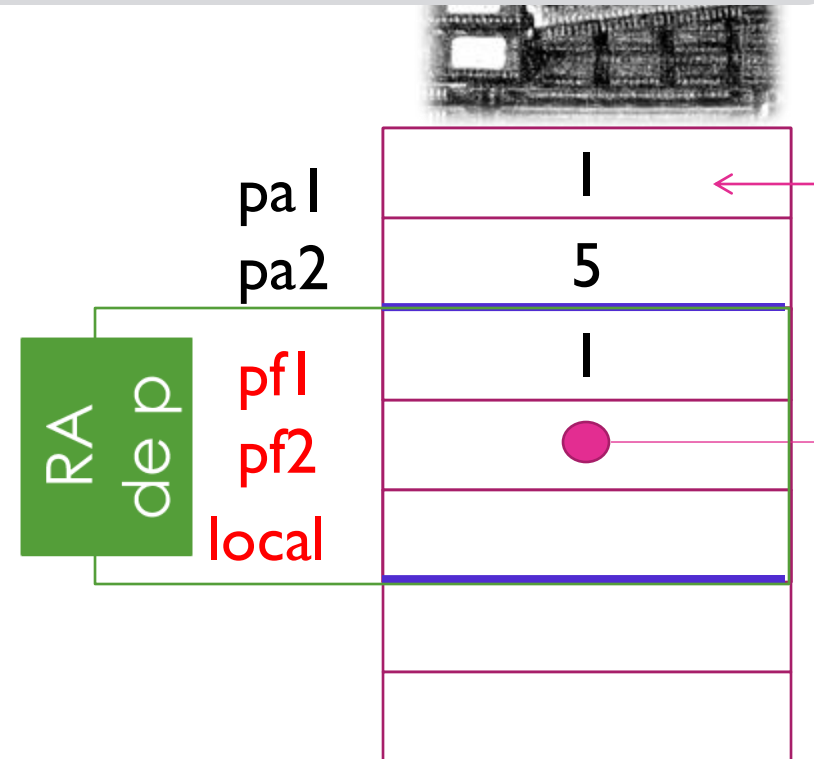


pa1	1
pa2	5

Manejo de memoria

```
program paramProc;  
var pa1, pa2:integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local := pf1+pf2;  
  pf1:= pf1+10; pf2 :=local+1;  
  writeln(pf1, pf2, local);  
end;  
begin  
  pa1:=1; pa2:=5;  
  p(pa1, pa1);  
  writeln(pa1, pa2);  
end.
```

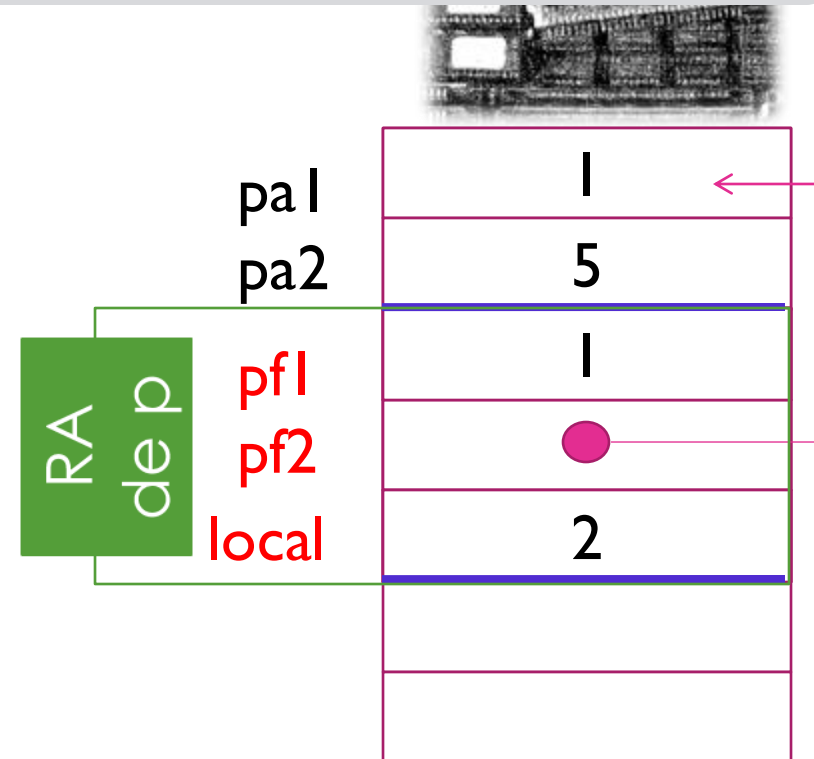
Flujo de ejecución



Manejo de memoria

```
program paramProc;  
var pa1, pa2:integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
→ local := pf1+pf2;  
  pf1:= pf1+10; pf2 :=local+1;  
  writeln(pf1, pf2, local);  
end;  
begin  
  pa1:=1; pa2:=5;  
  p(pa1, pa1);  
  writeln(pa1, pa2);  
end.
```

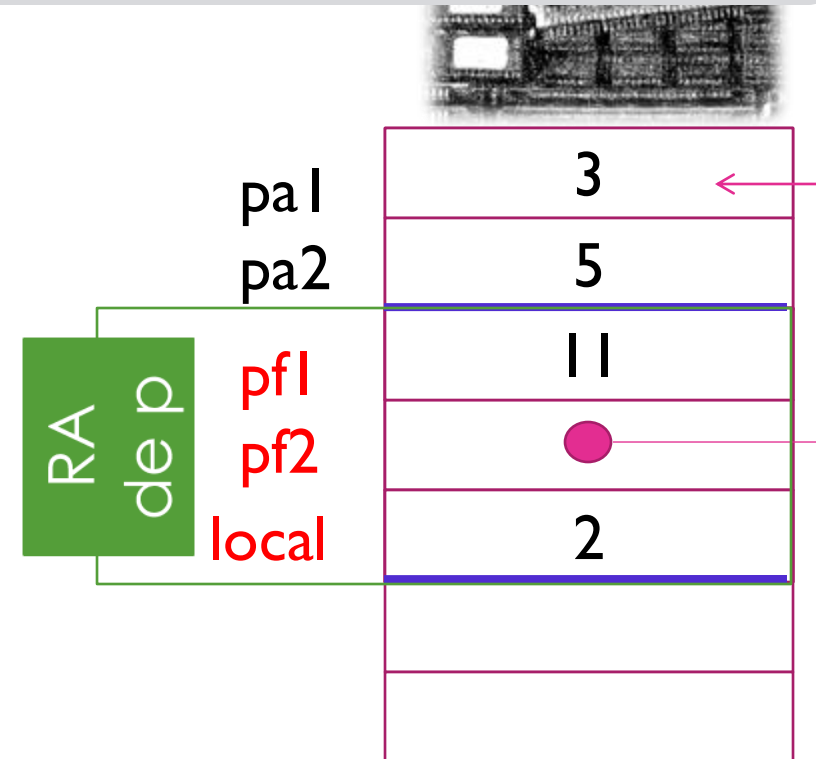
Flujo de ejecución



Manejo de memoria

```
program paramProc;  
var pa1, pa2:integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    local := pf1+pf2;  
    pf1:= pf1+10; pf2 :=local+1;  
    writeln(pf1, pf2, local);  
end;  
begin  
    pa1:=1; pa2:=5;  
    p(pa1, pa1);  
    writeln(pa1, pa2);  
end.
```

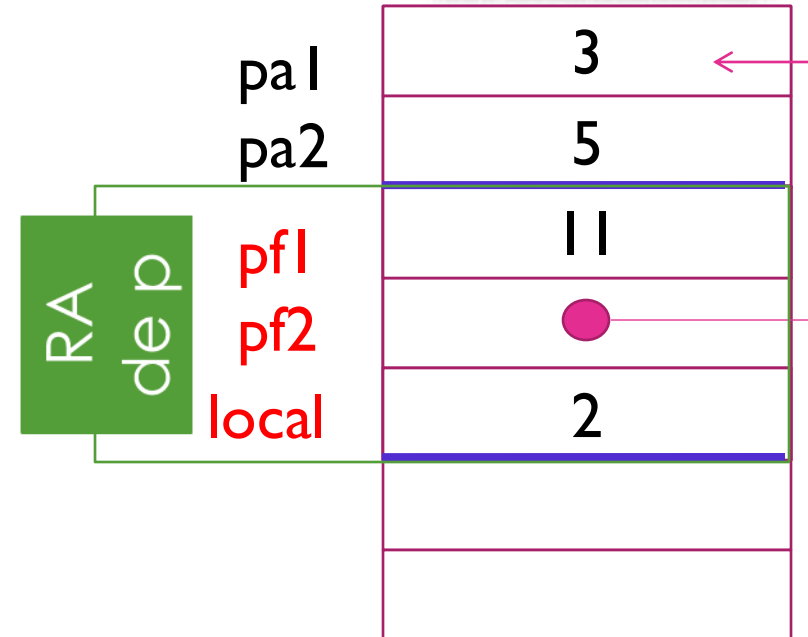
Flujo de ejecución



Manejo de memoria

```
program paramProc;  
var pa1, pa2:integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    local := pf1+pf2;  
    pf1:= pf1+10; pf2 :=local+1;  
    → writeln(pf1, pf2, local);  
end;  
begin  
    pa1:=1; pa2:=5;  
    p(pa1, pa1);  
    writeln(pa1, pa2);  
end.
```

Flujo de ejecución



11 3 2

Manejo de memoria

Flujo de ejecución

```
program paramProc;  
var pa1, pa2:integer;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    local := pf1+pf2;  
    pf1:= pf1+10; pf2 :=local+1;  
    writeln(pf1, pf2, local);  
end;  
begin  
    pa1:=1; pa2:=5;  
    p(pa1, pa1);  
→ writeln(pa1, pa2);  
end.
```



pa1
pa2

3
5

1132
35

Manejo de memoria

```
program paramProc;  
var pa1, pa2:integer;  
procedure p(var pf1: integer; var pf2: integer);
```

```
var local:integer;  
begin
```

Diferencia: Primer parámetro también por variable

```
    local := pf1+pf2;  
    pf1:= pf1+1; pf2 :=local+1;  
    writeln(pf1, pf2, local);
```

```
end;
```

```
begin
```

```
    pa1:=1; pa2:=5;  
    p(pa1, pa1);  
    p(pa2, pa2);
```

```
end.
```

pa1
pa2



Manejo de memoria

```
program paramProc;  
var pa1,pa2:integer;  
procedure p(pf1:integer; pf2:integer);  
var local:integer;  
begin  
    local := pf1+pf2;  
    pf1:= pf1+1; pf2 :=local+1;  
    writeln(pf1, pf2, local);  
end;  
begin  
    pa1:=1; pa2:=5;  
    p(pa1, pa1);  
end.
```

Diferencia: Los dos
parametros por valor

pa1
pa2



Manejo de memoria

```
program parampyf;  
var pa1, pa2: integer;  
function f(x: integer): integer;  
begin  
    f:=x+1;  
end;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
Begin  
    local := f(pf1) + f(pf2);  
    pf1:= pf1+10; pf2 :=local+1;  
    writeln(pf1, pf2, local);  
End;  
Begin  
    pa1:=1; pa2:=5;  
    p(pa1, pa2);  
    writeln(pa1, pa2);  
End.
```



pa1
pa2

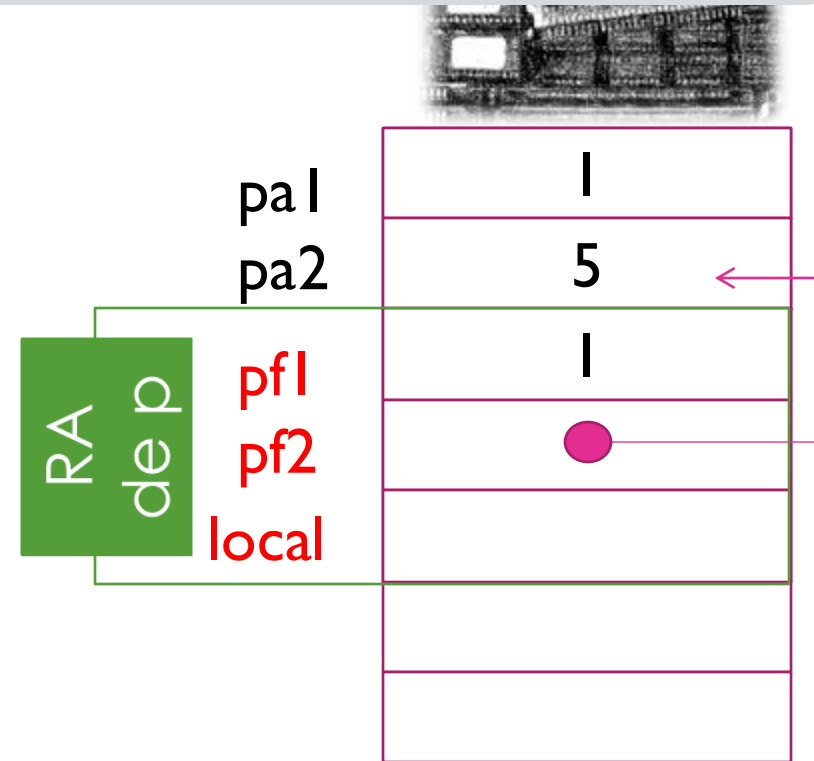
1
5

Se agrega la definición de una función

Manejo de memoria

```
program parampyf;  
var pa1, pa2: integer;  
function f(x: integer): integer;  
begin  
  f:=x+1;  
end;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local := f(pf1) + f(pf2);  
  pf1:= pf1+10; pf2 :=local+1;  
  writeln(pf1, pf2, local);  
end;  
begin  
  pa1:=1; pa2:=5;  
  p(pa1, pa2); { SUSPENDIDO }  
  writeln(pa1, pa2);  
end.
```

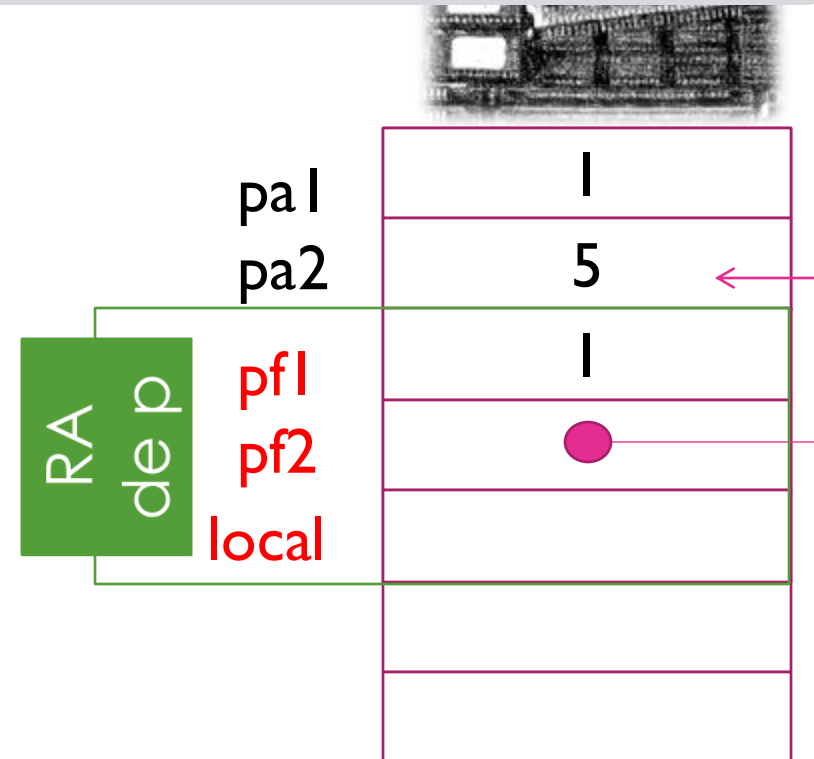
Flujo de ejecución



Manejo de memoria

```
program parampyf;  
var pa1, pa2: integer;  
function f(x: integer): integer;  
begin  
  f:=x+1;  
end;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local := f(pf1) + f(pf2);  
  pf1:= pf1+10; pf2 :=local+1;  
  writeln(pf1, pf2, local);  
end;  
begin  
  pa1:=1; pa2:=5;  
  p(pa1, pa2); { SUSPENDIDO }  
  writeln(pa1, pa2);  
end.
```

Flujo de ejecución

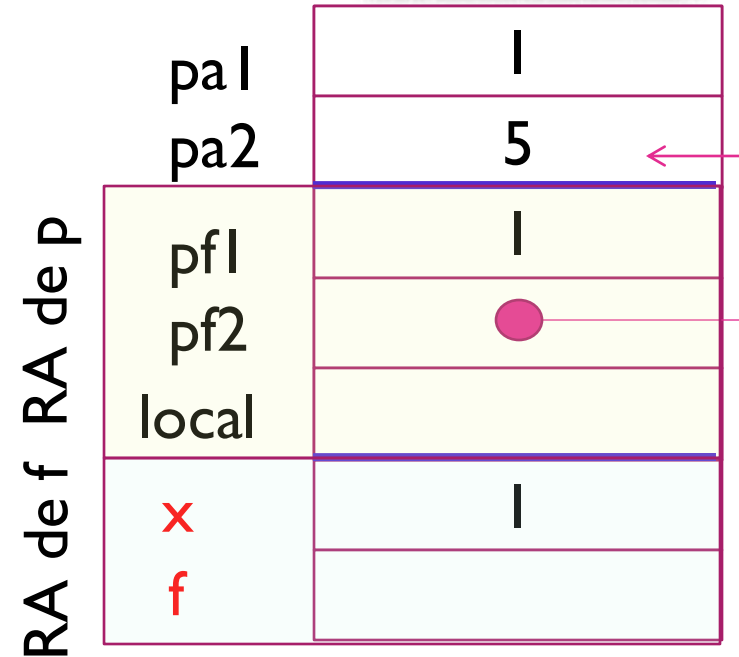


Manejo de memoria

```

program parampyf;
var pa1, pa2: integer;
function f(x: integer): integer;
begin
    f:=x+1;
end;
procedure p(pf1: integer; var pf2: integer);
var local: integer;
begin
    local := f(pf1) + f(pf2); { SUSPENDIDO }
    pf1:= pf1+10; pf2 :=local+1;
    writeln(pf1, pf2, local);
end;
begin
    pa1:=1; pa2:=5;
    p(pa1, pa2); { SUSPENDIDO }
    writeln(pa1, pa2);
end.
    
```

Flujo de ejecución



RA= Registro de activación

ARMADO del Registro de activación de una función:

- Se guarda una celda para cada parámetro formal.
- Se guarda una celda para cada variable local.
- Se guarda una celda para el nombre de la función.

PASAJE DE PARAMETROS

- En la celda correspondiente a cada parámetro formal POR VALOR se pone el valor del parámetro real asociado (buscar en la invocación).
- En la celda correspondiente a cada parámetro formal POR REFERENCIA se pone una referencia a la dirección del parámetro real asociado (buscar en la invocación).

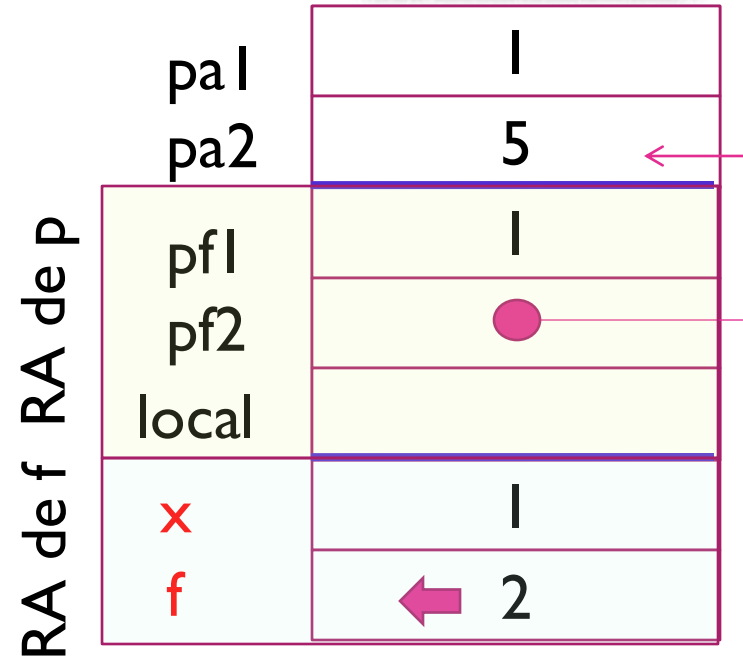
**SIEMPRE
IGUAL**

Manejo de memoria

```

program parampyf;
var pa1, pa2: integer;
function f(x: integer): integer;
begin
    f:=x+1;
end;
procedure p(pf1: integer; var pf2: integer);
var local: integer;
begin
    local := f(pf1) + f(pf2); { SUSPENDIDO }
    pf1:= pf1+10; pf2 :=local+1;
    writeln(pf1, pf2, local);
end;
begin
    pa1:=1; pa2:=5;
    p(pa1, pa2); { SUSPENDIDO }
    writeln(pa1, pa2);
end.
    
```

Flujo de ejecución



Cuando se llega al **end** de la función...

Flujo de ejecución

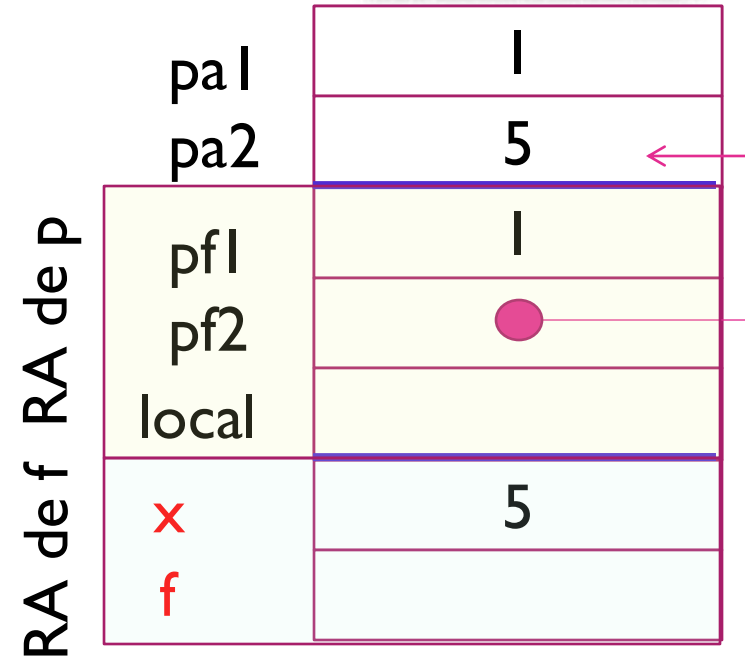


Manejo de memoria

```

program parampyf;
var pa1, pa2: integer;
function f(x: integer): integer;
begin
    f:=x+1;
end;
procedure p(pf1: integer; var pf2: integer);
var local: integer;
begin
    2
    local := f(pf1) + f(pf2); { SUSPENDIDO }
    pf1:= pf1+10; pf2 :=local+1;
    writeln(pf1, pf2, local);
end;
begin
    pa1:=1; pa2:=5;
    p(pa1, pa2); { SUSPENDIDO }
    writeln(pa1, pa2);
end.
    
```

Flujo de ejecución



Manejo de memoria

```

program parampyf;
var pa1, pa2: integer;
function f(x: integer): integer;
begin
  f:=x+1;
end;
procedure p(pf1: integer; var pf2: integer);
var local: integer;
begin
  local := f(pf1) + f(pf2); { SUSPENDIDO }
  pf1:= pf1+10; pf2 :=local+1;
  writeln(pf1, pf2, local);
end;
begin
  pa1:=1; pa2:=5;
  p(pa1, pa2); { SUSPENDIDO }
  writeln(pa1, pa2);
end.
  
```

Flujo de ejecución



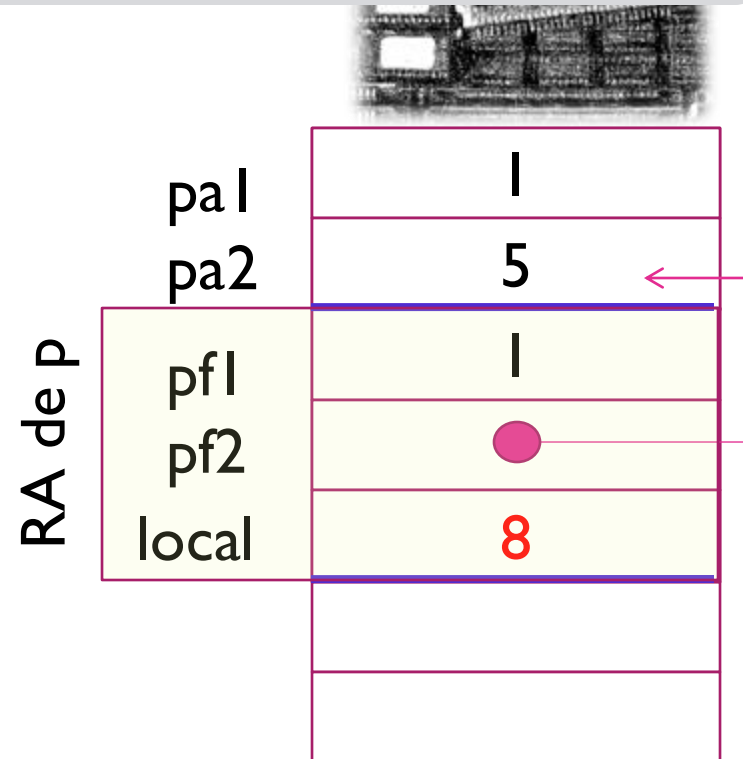
RA de f RA de p

pa1	1
pa2	5
pf1	1
pf2	●
local	
x	5
f	6

Manejo de memoria

```
program parampyf;  
var pa1, pa2: integer;  
function f(x: integer): integer;  
begin  
  f:=x+1;  
end;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  2      6  
  local := f(pf1) + f(pf2);  
  pf1:= pf1+10; pf2 :=local+1;  
  writeln(pf1, pf2, local);  
end;  
begin  
  pa1:=1; pa2:=5;  
  p(pa1, pa2); { SUSPENDIDO }  
  writeln(pa1, pa2);  
end.
```

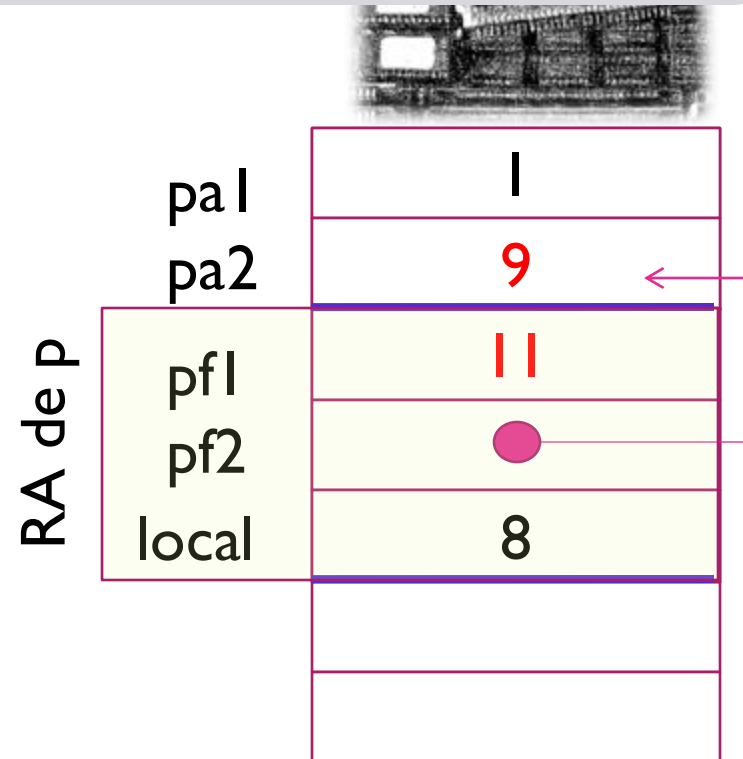
Flujo de ejecución



Manejo de memoria

```
program parampyf;  
var pa1, pa2: integer;  
function f(x: integer): integer;  
begin  
  f:=x+1;  
end;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local := f(pf1) + f(pf2);  
  pf1:= pf1+10; pf2 :=local+1;  
  writeln(pf1, pf2, local);  
end;  
begin  
  pa1:=1; pa2:=5;  
  p(pa1, pa2); { SUSPENDIDO }  
  writeln(pa1, pa2);  
end.
```

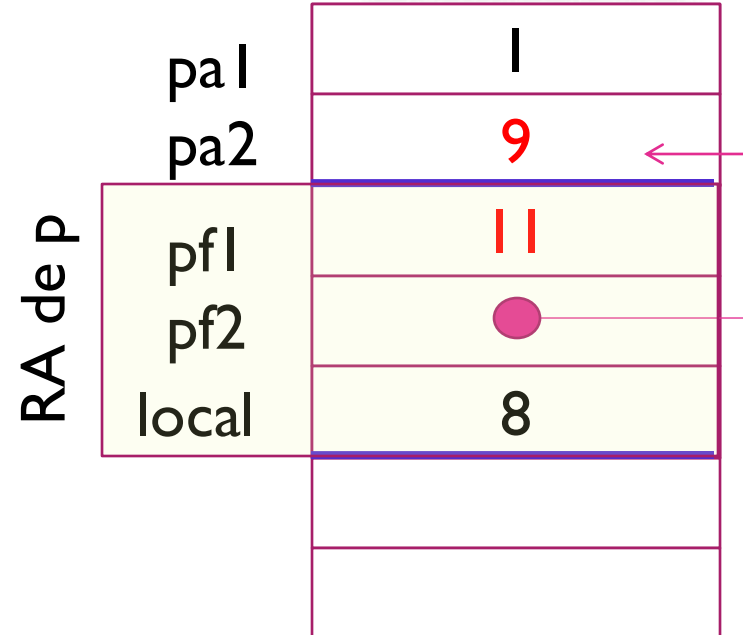
Flujo de ejecución



Manejo de memoria

```
program parampyf;  
var pa1, pa2: integer;  
function f(x: integer): integer;  
begin  
  f:=x+1;  
end;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
  local := f(pf1) + f(pf2);  
  pf1:= pf1+10; pf2 :=local+1;  
  →writeln(pf1, pf2, local);  
end;  
begin  
  pa1:=1; pa2:=5;  
  p(pa1, pa2); { SUSPENDIDO }  
  writeln(pa1, pa2);  
end.
```

Flujo de ejecución



11 9 8

Manejo de memoria

```
program parampyf;  
var pa1, pa2: integer;  
function f(x: integer): integer;  
begin  
  f:=x+1;  
end;  
procedure p(pf1: integer; var pf2 :integer);  
var local: integer;  
begin  
  local := f(pf1) + f(pf2);  
  pf1:= pf1+10; pf2 :=local+1;  
  writeln(pf1, pf2, local);  
end;  
begin  
  pa1:=1; pa2:=5;  
  p(pa1, pa2);  
→ writeln(pa1, pa2);  
end.
```

Flujo de ejecución



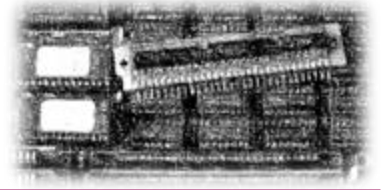
pa1
pa2

1
9

1198
19

Manejo de memoria

```
program parampyf;  
var pa1, pa2: integer;  
function f(x: integer): integer;  
begin  
    f:=x+1;  
end;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    local := f(pf1) + f(pf2);  
    pf1:= pf1+10; pf2 :=local+1;  
    writeln(pf1, pf2, local);  
end;  
begin  
    pa1:=1; pa2:=f(pa1);  
    p(pa1, pa2);  
    writeln(pa1, pa2);  
end.
```



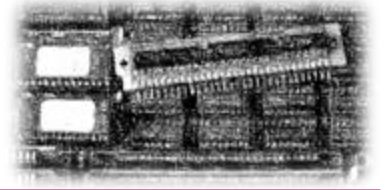
pa1
pa2

1



Manejo de memoria

```
program parampyf;  
var pa1, pa2: integer;  
function f(x: integer): integer;  
begin  
    f:=x+1;  
end;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    local := f(pf1) + f(pf2);  
    pf1:= pf1+10; pf2 :=local+1;  
    writeln(pf1, pf2, local);  
end;  
begin  
    pa1:=1; pa2:=f(pa1);  
    p(f(pa1*2), pa2);  
    writeln(pa1, pa2);  
end.
```



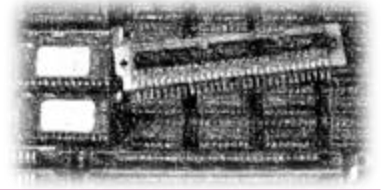
pa1
pa2

1
2



Manejo de memoria

```
program parampyf;  
var pa1, pa2: integer;  
function f(x: integer): integer;  
begin  
    f:=x+1;  
end;  
procedure p(pf1: integer; var pf2: integer);  
var local: integer;  
begin  
    local := f(pf1) + f(pf2);  
    pf1:= pf1+10; pf2 :=local+1;  
    writeln(pf1, pf2, local);  
end;  
begin  
    pa1:=1; pa2:=f(pa1);  
    p(f(pa1*2), f(pa2));  
    writeln(pa1, pa2);  
end.
```



pa1
pa2

Error en compilación
El parámetro real no puede ser
una llamada a función cuando el
FORMAL es por referencia

Manejo de memoria



Dibujar la memoria al momento del **Begin** del bloque ejecutable de f.

```
Program progMEM;  
  Var a: _ ; b: _ ;  
  Function f(x, y: _ ): _ ;  
    Begin  
    ...  
  End;  
  Procedure p(z1: _ ; var z2: _);  
    Begin  
      z2 := f(z1+20, z1*5);  
    End;  
    Begin  
      a:=10;  
      p(a, b);  
    ...  
  End.
```

Manejo de memoria



Dibujar la memoria al momento del **Begin** del bloque ejecutable de P.

```
Program MEMO;
```

```
Var a, b, c, d: integer;
```

```
Procedure P(x1: integer; var x2, x3: integer);
```

```
Var z: integer;
```

```
Begin
```

```
...
```

```
End;
```

```
Begin
```

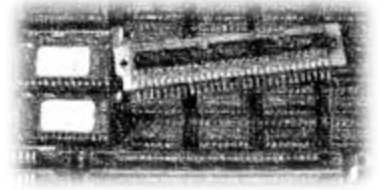
```
a:=2; b:=5;
```

```
P(a+b, c, d);
```

```
...
```

```
End.
```

Manejo de memoria

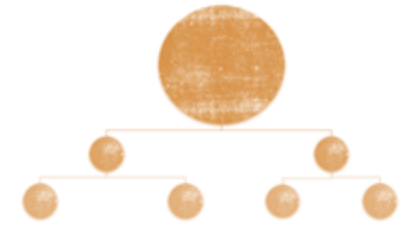


¿TE ANIMAS A COMPLETAR LOS ' _ '?

```
Program progMEM;  
  Var a: _ ; b: _ ;  
  Function f(x, y: _ ): _ ;  
  Begin  
    f := x > y;  
  End;  
  Procedure p(z1: _ ; var z2: _ );  
  Begin  
    z2 := f(z1+20, z1*5);  
  End;  
  Begin  
    a:=10;  
    p(a, b);  
    ...  
  End.
```

a	10
b	
z1	10
z2	⊙
x	30
y	50
f	

Procedimientos y Funciones

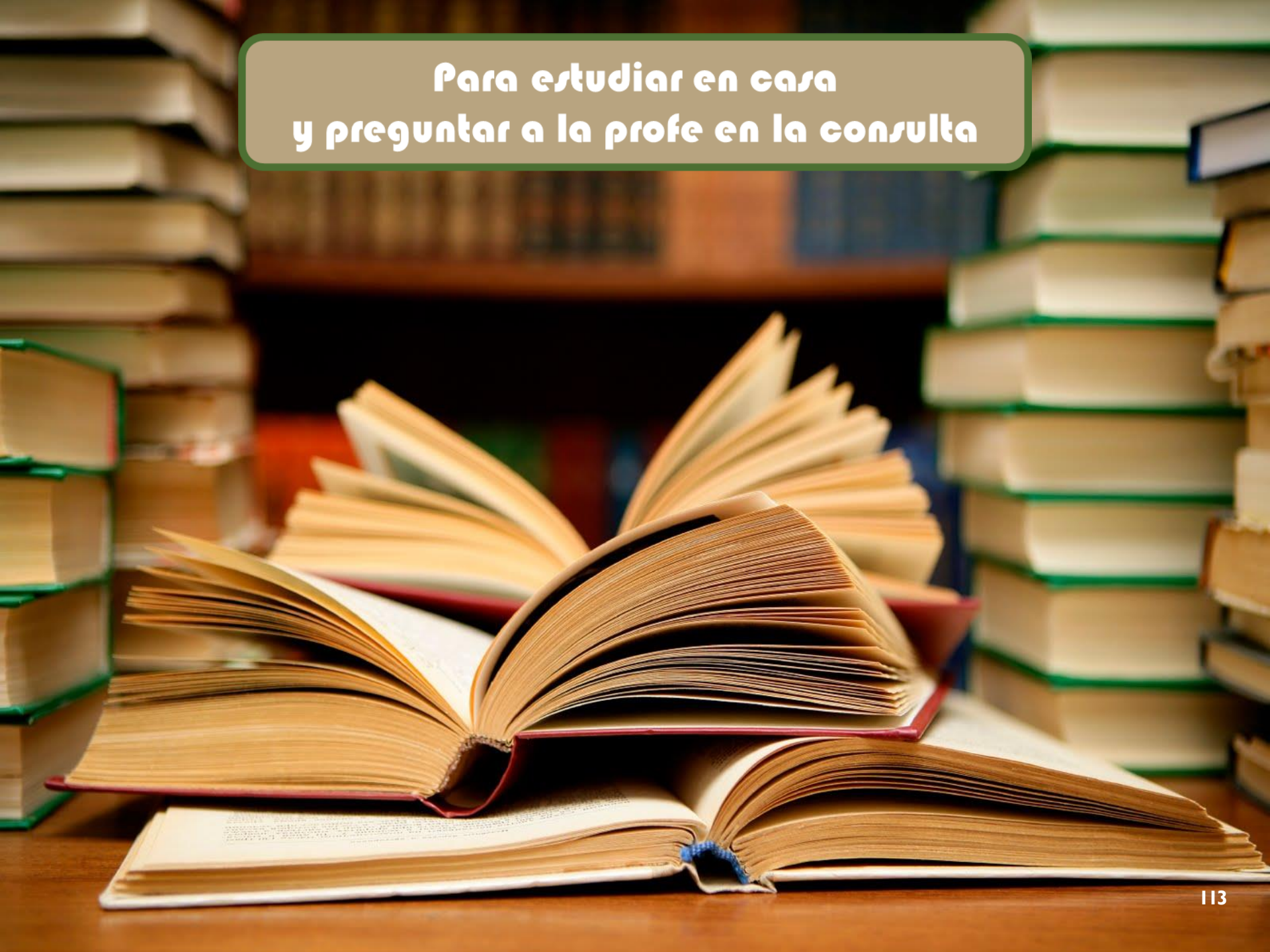


Problema: Leer una secuencia de números positivos y mostrar los que son capicúa.

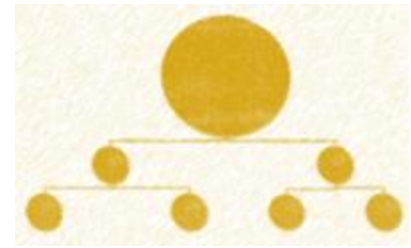
Escriba una función que determine si un número es capicúa y un programa que usando esta primitiva resuelva el problema.



**Para estudiar en casa
y preguntar a la profe en la consulta**



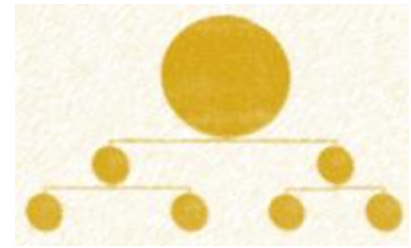
Procedimientos



La memoria puede visualizarse como una secuencia de celdas cada una de las cuales tiene asociada una dirección y un contenido.

dirección	contenido
↓	↓
0000	0011001
0001	1101000
0010	1110110
0011	1100001
0100	0111101
0101	1100110

Procedimientos



La memoria puede visualizarse como una secuencia de celdas cada una de las cuales tiene asociada una dirección y un contenido.

dirección	contenido
↓	↓
0000	00 11001
0001	11 01000
0010	11 10110
0011	11 0 000 1
0100	0 111101
0101	11 00110

Almacena LA DIRECCION de otra celda

Parámetro por referencia

Program pyf;
var a, b, c: integer;

Definición: Variable global es aquella variable declarada en el PROGRAMA PRINCIPAL.

procedure sigant(n: integer; var ant, sig: integer);
begin
 ant := n - 1;
 sig := n + 1;
end;

Son dos datos distintos
e independientes.

function doble(n: integer): integer;
begin
 doble := n * 2;
end;

Begin
 write ('Ingrese un número');
 readln(a);
 sigant(a, b, c);
 a := doble(b) + doble(c);
 sigant(a, c, b);

End.

**funciones
y
procedimientos**

Bloque ejecutable del programa principal: sólo puede acceder a las variables globales.

Procedimientos y Funciones

funciones
y
procedimientos

```
procedure sigant(n: integer; var ant, sig: integer);  
begin  
    ant := n - 1;  
    sig := n + 1;  
end;
```

El procedimiento **sigant** tiene tres parámetros.

El parámetro **n** es el dato de entrada y está pasado por valor.

Cuando empieza la ejecución del procedimiento, se reserva una locación de memoria para **n**, que se inicializa con el valor del parámetro real y se destruye cuando termina el procedimiento.

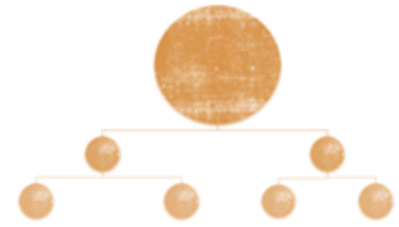
También se reservan locaciones para **sig** y **ant**. Los parámetros **ant** y **sig** son datos de salida y se definen como parametros por referencia o por variable. **Son referencias a las locaciones de memoria que corresponden a los parámetros reales.**



Procedimientos y Funciones



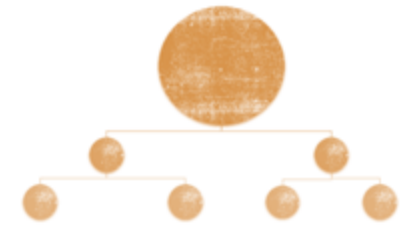
```
function doble(n: integer): integer;  
begin  
  doble := n * 2;  
end;
```



La función doble tiene un único parámetro, es un dato de entrada pasado por valor.

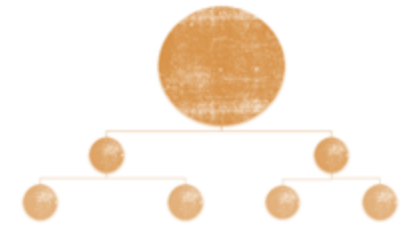
La función computa un valor a partir de n y lo asigna a su nombre.

Procedimientos y Funciones



```
Program pyf;  
var a, b, c: integer;  
...  
Begin  
  write ('Ingrese un número');  
  readln(a);  
  sigant(a, b, c);  
  a := doble(b) + doble(c);  
  sigant(a, c, b);  
...  
End.
```

Procedimientos y Funciones



```
procedure sigant (n: integer; var ant, sig: integer);  
begin  
    ant := n - 1;  
    sig := n + 1;  
end;
```

sigant(a, b, c);

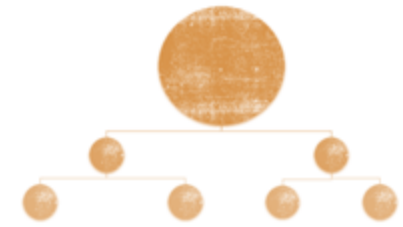
La primera invocación de sigant asigna el valor del parámetro real a al parámetro formal n.

El parámetro real b queda ligado al parámetro formal ant, **cualquier modificación en ant, afecta al valor almacenado en b.**

El parámetro real c queda ligado al parámetro formal sig, **cualquier modificación en sig, afecta al valor almacenado en c.**

Cuando termina la ejecución se destruyen las locaciones de memoria ligadas a los parámetros.

Procedimientos y Funciones



```
function doble(n : integer): integer;  
begin  
  doble := n * 2;  
end;
```

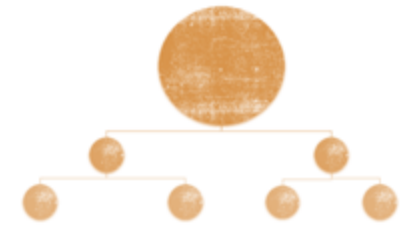
```
a:= doble(b) + doble (c);
```

En la **primera** invocación a `doble`, se reserva una locación de memoria para variable local `n`, y se inicializa con el valor del parámetro real `b`.

Cuando la función se termina la locación se destruye y se reanuda la evaluación de la expresión.

El resultado de la primera invocación constituye el valor del primer operando de la suma.

Procedimientos y Funciones



```
function doble(n : integer): integer;  
begin  
  doble := n * 2;  
end;
```

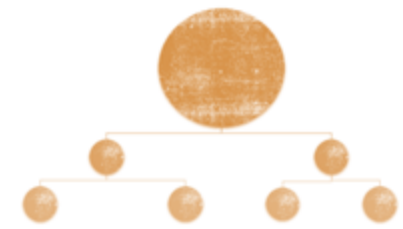
```
a:= doble(b) + doble (c);
```

En la **segunda** invocación a **doble**, se reserva una locación de memoria para variable local **n**, y se inicializa con el valor del parámetro real **c**.

Cuando la función se termina la locación se destruye y se reanuda la evaluación de la expresión.

El resultado de la segunda invocación constituye el valor del segundo operando de la suma.

Procedimientos y Funciones



```
procedure sigant (n: integer; var ant, sig: integer);  
begin  
    ant := n - 1;  
    sig := n + 1;  
end;  
  
sigant(a, c, b);
```

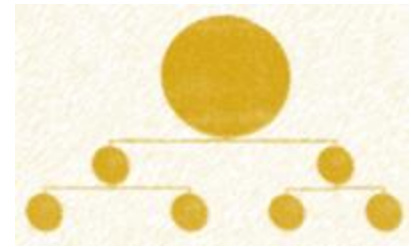
La segunda invocación de sigant asigna el valor del parámetro real a al parámetro formal n.

El parámetro real c queda ligado al parámetro formal ant, cualquier modificación en ant, afecta al valor almacenado en c.

El parámetro real b queda ligado al parámetro formal sig, cualquier modificación en sig, afecta al valor almacenado en b.

Cuando termina la ejecución se destruyen las locaciones de memoria ligadas a los parámetros.

Procedimientos



Un parámetro por valor se inicializa en el momento de la invocación del procedimiento con el valor del parámetro real.

Al terminar la ejecución del procedimiento la variable asociada al parámetro formal se destruye y su valor se pierde.

Un parámetro por referencia introduce un nuevo nombre para referirse a la celda de memoria que corresponde al parámetro real.

Ejercicio: Hacer la traza completa asumiendo que se ingresa un 7 (para la instrucción *readln(a)* del programa principal).

Procedimientos y Funciones



ACTIVACIÓN DE UN PROCEDIMIENTO

- Una invocación a **procedimiento** es una **instrucción**.
- Una vez invocado, **el control** (la ejecución) **pasa al procedimiento**.
- El **bloque de código que incluye la invocación, se suspende**.
- Cuando comienza la ejecución se reserva espacio en memoria para las variables locales y para los parámetros formales por valor, que se inicializan con los **valores** de los parámetros reales. También para los parámetros formales por variable que almacenan la dirección de los parámetros reales correspondientes.

Procedimientos y Funciones



ACTIVACIÓN DE UN PROCEDIMIENTO

- El procedimiento se ejecuta y cuando termina, el control retorna, provocando algún efecto.
- El efecto de la ejecución de un procedimiento puede ser modificar uno o más parámetros por variable o producir una salida por consola o por archivo.
- El bloque de código que había quedado suspendido se reanuda, en la instrucción siguiente a la invocación.

Procedimientos y Funciones



ACTIVACIÓN DE UNA FUNCIÓN

- Una **función** se invoca desde una **expresión**.
- El bloque de código en donde aparece la invocación a la función, se suspende.
- El control (la ejecución) pasa al bloque ejecutable de la función.
- En ese momento, antes de ejecutar la primera instrucción de la función, se reserva espacio en memoria para las variables locales y para los parámetros por valor, que se inicializan con los valores de los parámetros reales. También se reserva un espacio para el resultado de la función.

Procedimientos y Funciones



ACTIVACIÓN DE UNA FUNCIÓN

- La función se ejecuta y cuando termina, el control retorna al lugar de la invocación, continuando con la evaluación de la expresión que la contiene.
- El efecto de la ejecución de una función **DEBERIA SER SOLAMENTE** retornar un resultado.